

Olas

Smart Contract Security Assessment

Audit dates: Jan 22 — Feb 09, 2026

Overview

About C4

Code4rena (C4) is a competitive audit platform where security researchers, referred to as Wardens, review, audit, and analyze codebases for security vulnerabilities in exchange for bounties provided by sponsoring projects.

During the audit outlined in this document, C4 conducted an analysis of the Olas smart contract system. The audit took place from January 22 to February 09, 2026.

Final report assembled by Code4rena.

Summary

The C4 analysis yielded an aggregated total of 23 unique vulnerabilities. Of these vulnerabilities, 11 received a risk rating in the category of HIGH severity and 12 received a risk rating in the category of MEDIUM severity.

Additionally, C4 analysis included 28 QA reports compiling issues with a risk rating of LOW severity or informational.

All of the issues presented here are linked back to their original finding, which may include relevant context from the judge and Olas team.

Scope

The code under review can be found within the [C4 Olas repository](#), and is composed of 40 smart contracts written in the Solidity programming language.

The code in C4's Olas repository was pulled from:

- Repository:
 - [Governance](#)
 - [Tokenomics](#)
 - [Registries](#)
- Commit hash:
 - Governance: 6ea160398200a7b1ca6bf916fc2cd91483bff707
 - Tokenomics: bbec5ac12721a62672fb7a5ffba5c40f5a46d8cb
 - Registries: be1057a5e37f17f26b13c41311fe0e8e40259484

Severity Criteria

C4 assesses the severity of disclosed vulnerabilities based on three primary risk categories: high, medium, and low/informational.

High-level considerations for vulnerabilities span the following key areas when conducting assessments:

- Malicious Input Handling
- Escalation of privileges
- Arithmetic
- Gas use

For more information regarding the severity criteria referenced throughout the submission review process, please refer to the documentation provided on [the C4 website](#), specifically our section on [Severity Categorization](#).

High Risk Findings (11)

[\[H-01\] Broken TWAP validation allows spot-price manipulation and renders slippage checks ineffective](#)

Submitted by [BenRai](#), also found by [Ox_DyDx](#), [Oxastronatey](#), [Oxbrett8571](#), [OxDemon](#), [OxFBI](#), [Oxkujen](#), [Oxshdax](#), [OxVrka](#), [4nibhal](#), [7Tecra7](#), [Aamir](#), [ACai](#), [AcsonBarbosa](#), [AgentPulse](#), [aman234](#), [AnonOpcode](#), [Auditor3](#), [ayazwx](#), [bhatmuneeb](#), [Biqua](#), [BlueSheep](#), [bun](#), [catower917](#), [ChainSentry](#), [chuvak](#), [codecoffeeguy](#), [coinsspor](#), [count-sum](#), [Cryptor](#), [DARKNAVY](#), [dexter](#), [Dillon](#), [dman](#), [DQH1](#), [Egbe](#), [emerald7017](#), [Funen](#), [fx](#), [Gakarot](#), [gaving101](#), [golem0](#), [grearlake](#), [happykilling](#), [harry](#), [hodlturk](#), [Huang](#), [humaira45](#), [ibrahimatix0x01](#), [ix2b100](#), [jayx](#), [jerry0422](#), [jkk812812](#), [jo13](#), [JoshuaM33](#), [JuggerNaut63](#), [justingoro](#), [K42](#), [kind0dev](#), [KKKKK](#), [kmmk](#), [konstantinos193](#), [Konzy](#), [KuwaTakushi](#), [Lamsy](#), [MaroOoOo](#), [maxcrate](#), [Morraez](#), [mrMorningstar](#), [MRSHEEP](#), [Nakamoto](#), [Ni9htNi9ht](#), [nisedo](#), [Noahdgoat](#), [Nyx](#), [odeili](#), [OuttakeClaw](#), [PlayboiEvokid](#), [prajithnair98](#), [recr4sh](#), [Reey](#), [richa](#), [Rifter](#), [rzizah](#), [saeedfesa10](#), [sahuang](#), [sanbir](#), [santipu](#), [shalevhvs](#), [Silvermist](#), [sruibm](#), [suhyeon](#), [taronsung](#), [tedre](#), [thedream](#), [tradingview](#), [uwezkhano6](#), [UzeyirCh](#), [valarislife](#), [Valves](#), [Valves](#), [Varun_05](#), [Viridis](#), [vitaliishuba](#), [white-fox02](#), [Wojack](#), [wonderofme](#), [y4y](#), [Zbugs](#), [zeroDayHunter](#), [zpan](#), and [zzebra83](#)

`autonolas-tokenomics/.../contracts/oracles/UniswapPriceOracle.sol` [#L55-L91](#)

The `validatePrice()` function does not compute a real Uniswap V2 TWAP and instead collapses to the current spot price, making the slippage check ineffective and trivially manipulable within a single block.

Finding Description

The `validatePrice()` function is meant to validate the trade price against a time-weighted average price (TWAP) from a Uniswap V2 pair, enforcing a maximum allowed slippage.

Correct TWAP-based validation is a core security mechanism against flash loans, sandwich attacks, and short-term price manipulation by averaging prices over time.

Uniswap V2 exposes cumulative price variables

(`price0CumulativeLast` and `price1CumulativeLast`) that grow monotonically over time, encoding the reserve ratio in UQ112x112 format multiplied by elapsed time. To compute a TWAP, a consumer must:

1. Snapshot cumulative prices and timestamps at time `t0`
2. Read cumulative prices again at time `t1`
3. Compute the average price as the difference divided by elapsed time

`validatePrice()` deviates from this design:

- It reads the latest cumulative price from the pair
- Reads `blockTimestampLast` from `getReserves()`
- Builds an artificial “cumulative price” using the current `tradePrice`
- Derives a “TWAP” from these values
- Compares that “TWAP” to the current trade price to enforce slippage

The core bug is in the construction and use of cumulative prices:

```
uint256 cumulativePrice = cumulativePriceLast + (tradePrice * elapsedTime);

uint256 timeWeightedAverage = (cumulativePrice -
    cumulativePriceLast) / elapsedTime;
```

This simplifies to

```
timeWeightedAverage =
(cumulativePriceLast + (tradePrice * elapsedTime) - cumulativePriceLast)
/ elapsedTime =
(tradePrice * elapsedTime) / elapsedTime =
tradePrice
```

The computed TWAP is always equal to the current `tradePrice`, regardless of history, elapsed time, or pool liquidity. The slippage check is effectively nullified because the deviation is always zero (aside from rounding).

Further issues:

- `tradePrice` comes from `getPrice()` and is not guaranteed to reflect Uniswap reserves.

- The Uniswap V2 cumulative prices are $UQ112 \times 112$, but the function uses raw uint256 math without proper fixed-point handling.
- `blockTimestampLast` reflects the last reserve update, not a consumer-controlled oracle observation.
- The function is view and uses no persisted state, so it is fully exploitable via flash loans or same-block reserve manipulation.

UniswapPriceOracle is used on chains with Uniswap V2 pools (e.g. Ethereum mainnet). `validatePrice()` is the gate for all OLAS swap and liquidity-exit flows that use V2 oracles.

1. `BuyBackBurner._buyOLAS()` (V2 path)

- Called from `buyBack(secondToken, secondTokenAmount)` when using V2 pools.
- Flow: `require(!Oracle(poolOracle).validatePrice(maxSlippage), "Before swap slippage limit is breached")` → `getPrice()` → `_performSwap()` → `getPrice()` again → post-swap bounds check.
- Anyone can call `buyBack()` after secondTokens (e.g. WETH) are deposited into the contract. The protocol swaps treasury/user deposits for OLAS and sends it to `bridge2Burner` for burning or bridging.
- Because `validatePrice()` always passes ($TWAP \approx \text{spot}$), the pre-swap check is ineffective. An attacker can flash-loan, manipulate the pool, and have the buyback execute at the manipulated price. The protocol receives less OLAS per unit of ETH, and the attacker can unwind to capture arbitrage profit.

2. `LiquidityManagerETH._checkTokensAndRemoveLiquidityV2()` and `LiquidityManagerOptimism._checkTokensAndRemoveLiquidityV2()`

- Called from `convertToV3()` when migrating V2 LP to V3.
- Flow: if `(!Oracle(oracleV2).validatePrice(maxSlippage / 100))` revert `SlippageLimitBreached()` → remove V2 liquidity → mint V3.
- Only the owner can call `convertToV3()`. The oracle is used to ensure the V2 exit happens at a fair price before migrating to V3.
- Because `validatePrice()` always passes, the protocol can exit V2 liquidity at a manipulated price. The resulting token amounts (OLAS and quote) are worse than the true TWAP would imply, and the attacker can profit from the price deviation when the protocol swaps or migrates.

Attacker scenario

1. Observe a `buyBack()` or `convertToV3()` transaction in the mempool.
2. Swap to move the Uniswap V2 pool price (e.g. large swap in either direction).
3. Execute the victim transaction in the same block.
4. `validatePrice()` passes because $TWAP \approx \text{manipulated spot}$.
5. Swap or liquidity removal executes at the manipulated price.

6. Attacker reverses the pool manipulation in the same block.
7. Attacker pockets the arbitrage; the protocol or users take the loss.

The attack is cheap (flash loan + gas), permissionless, and repeatable. The same logic applies to any chain where UniswapPriceOracle is used for OLAS swaps or liquidity exits.

Recommendation

Implement a proper Uniswap V2 TWAP oracle pattern by persisting cumulative price snapshots across blocks and computing averages from differences between observations:

- Store price{0,1}CumulativeLast and timestampLast in contract storage
- Update them in a state-changing function (e.g. updateOracle())
- Compute TWAPs using cumulative price deltas
- Correctly handle UQ112x112 fixed-point math

Slippage should be evaluated only after a correct TWAP is computed. This restores the intended oracle guarantees and protects against same-block and flash-loan-based price manipulation.

[H-02] Variable Overwrite in `checkPoolAndGetCenterPrice()` Creates Dead-Code Deviation Check, Leaving All V3 Protocol-Owned Liquidity Operations Unprotected

Submitted by [fx](#), also found by [Ox_oi](#), [Oxastronatey](#), [OxDemon](#), [OxFBI](#), [OxSeer](#), [abdelhaq16](#), [AcsonBarbosa](#), [AnonOpcode](#), [Aristos](#), [Auditor3](#), [Avalance](#), [DanXIII](#), [Egbe](#), [emerald7017](#), [ESG](#), [Funen](#), [Gakarot](#), [gesha17](#), [golemO](#), [grearlake](#), [humaira45](#), [I1iveF0rTh1Sh1t](#), [inh3l](#), [Inspektor](#), [jayx](#), [jkk812812](#), [jo13](#), [konstantinos193](#), [KuwaTakushi](#), [martila_iO](#), [mrdafidi](#), [mrMorningstar](#), [NexusAudits](#), [Nyx](#), [octOpwn](#), [odeili](#), [osok](#), [peazzycocle](#), [PlayboiEvokid](#), [rox_k](#), [rzizah](#), [s3ma4](#), [sahuang](#), [santipu_](#), [saraswati](#), [Silvermist](#), [udaykiranpedda](#), [Uhudsavasindankacanokcu2](#), [uwezkhan06](#), [Valves](#), [Varun_05](#), and [Yu4n](#)

- `autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol` [#L1091-L1137](#)
- `autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol` [#L1104-L1106](#)
- `autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol` [#L1113-L1116](#)
- `/autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol` [#L1119](#)
- `/autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol` [#L1123](#)

`LiquidityManagerCore.checkPoolAndGetCenterPrice()` is the sole TWAP oracle guard for all V3 protocol-owned liquidity operations. It protects **five LiquidityManager** functions that manage protocol-owned concentrated liquidity positions, plus two `BuyBackBurner` call sites:

LiquidityManager operations (protocol-owned liquidity):

- `convertToV3()` (line 689) — migrating V2 liquidity to V3
- `changeRanges()` (line 761) — adjusting concentrated liquidity tick ranges
- `collectFees()` (line 822) — collecting accrued trading fees
- `decreaseLiquidity()` (line 893) — removing liquidity from positions
- `increaseLiquidity()` (line 974) — adding liquidity to positions

BuyBackBurner call sites:

- `BuyBackBurner._buyOLAS()` (line 231) — V3 swap path
- `BuyBackBurner.checkPoolPrices()` (line 367) — legacy compatibility

The function is supposed to compare the pool's current spot price against a TWAP and revert if the deviation exceeds `MAX_ALLOWED_DEVIATION` (10%). However, a variable overwrite at line 1119 causes the deviation check to compare the TWAP against itself, producing a deviation of zero on every call. **The `MAX_ALLOWED_DEVIATION` check is dead code — it can never trigger.**

Additionally, two early-return paths bypass the deviation check entirely for stale or new pools, returning the raw spot price without any validation. Together, these bugs cover every possible execution path:

Root Cause A: Early-Return Bypasses (Lines 1104, 1114)

Two early-return paths skip TWAP validation entirely and return the raw spot price:

```
// PATH 1: Stale observations (line 1104-1106)
(uint32 oldestTimestamp,,) =
IUniswapV3(pool).observations(observationIndex);
if (oldestTimestamp + SECONDS_AGO < block.timestamp) {
    return centerSqrtPriceX96; // Returns SPOT price, NO TWAP check
}

// PATH 2: TWAP call failure (line 1113-1115)
(bool success, bytes memory returnData) =
address(this).staticcall(payload);
if (!success) {
    return centerSqrtPriceX96; // Returns SPOT price, NO TWAP check
}
```

Path 1 triggers when the pool's most recent observation is more than `SECONDS_AGO` (1800s = 30 min) old. This fires for any pool that hasn't had a swap in 30+ minutes.

Path 2 triggers when `getTwapFromOracle()` fails, which happens for newly created pools (e.g., immediately after `convertToV3()`) that don't have enough observation history for a 30-minute TWAP.

Both return the current spot price with no manipulation check.

Root Cause B: Return-Variable Overwrite at Line 1119 Makes Deviation Check Dead Code (Primary Bug)

When the function passes both early-return checks (active pool with sufficient history), the TWAP deviation check executes — but a variable overwrite causes it to compare the TWAP price against itself, producing zero deviation on every call. The mechanism is a reuse of the function's return variable `centerSqrtPriceX96` as the decode target for the TWAP result:

```
function checkPoolAndGetCenterPrice(address pool) public view returns
(uint160 centerSqrtPriceX96) {
    // Line 1094: centerSqrtPriceX96 = SPOT price from slot0
    (centerSqrtPriceX96, observationIndex) =
    _getPriceAndObservationIndexFromSlot0(pool);

    // ... early returns (Root Cause A) ...

    // Line 1119: centerSqrtPriceX96 is OVERWRITTEN with TWAP-derived
    sqrt price
    (twapPrice, centerSqrtPriceX96) = abi.decode(returnData, (uint256,
    uint160));
    //          ^^^^^^^^^^^^^^^^^^^^^ original spot price is LOST

    // Line 1123: "instant price" is computed from the OVERWRITTEN
    variable
    uint256 instantPrice = mulDiv(uint256(centerSqrtPriceX96),
    uint256(centerSqrtPriceX96), (1 << 64));
    // instantPrice == twapPrice (both derive from the same
    centerSqrtPriceX96)

    // Lines 1128-1130: deviation = |twapPrice - twapPrice| / twapPrice =
    0 ALWAYS
    deviation = (instantPrice > twapPrice)
        ? mulDiv((instantPrice - twapPrice), 1e18, twapPrice)
        : mulDiv((twapPrice - instantPrice), 1e18, twapPrice);

    // Line 1134: This NEVER triggers because deviation is always 0
    if (deviation > MAX_ALLOWED_DEVIATION) {
        revert Overflow(deviation, MAX_ALLOWED_DEVIATION);
    }
}
```

The precise mechanism: `centerSqrtPriceX96` is the function's named return variable, initialized to the spot price from `slot0()` at line 1094. At line 1119, `abi.decode(returnData, (uint256, uint160))` overwrites this variable with the TWAP-derived sqrt price from

`getTwapFromOracle()`. The `getTwapFromOracle()` function (line 1067-1086) returns (`price`, `centerSqrtPriceX96`) where $price = centerSqrtPriceX96^2 / 2^{64}$ — the two return values are mathematically coupled. When line 1123 then computes `instantPrice = centerSqrtPriceX96^2 / 2^{64}`, it reproduces the exact same computation that produced `twapPrice`, because `centerSqrtPriceX96` now holds the TWAP sqrt price instead of the spot sqrt price. The deviation is mathematically guaranteed to be zero:

```
instantPrice = overwritten_centerSqrtPriceX96^2 / 2^64
              = twap_sqrt^2 / 2^64
              = twapPrice                               (by definition from
getTwapFromOracle)
deviation    = |twapPrice - twapPrice| / twapPrice = 0    (ALWAYS)
```

Combined Effect

POOL STATE	CODE PATH	RESULT
Stale observations (>30 min since last swap)	Root Cause A, Path 1 (line 1104)	Returns spot price, no TWAP check
New pool (insufficient observation history)	Root Cause A, Path 2 (line 1114)	Returns spot price, no TWAP check
Active pool with sufficient history	Root Cause B (line 1119)	TWAP check runs but deviation = 0 always

There is no condition under which this function detects price manipulation.

Risk

Likelihood: Medium

Root Cause A requires a pool in a specific state (stale or new). This is realistic -- OLAS is not a high-frequency trading token, and 30+ minutes without a swap on a specific pool is common, especially on L2 chains. New pools are immediately vulnerable after `convertToV3()`.

Root Cause B fires on every call where the TWAP oracle succeeds -- the "normal" code path for active pools with sufficient history. This is the default condition for established pools.

Together, every possible pool state is covered. An attacker can monitor pool activity and act when conditions align (Root Cause A), or attack any active pool at any time (Root Cause B).

Impact: Medium

The `MAX_ALLOWED_DEVIATION` check is dead code — it cannot trigger under any condition. This leaves all five `LiquidityManager` operations exposed to flash-loan price manipulation:

- `changeRanges()` (line 761): Uses `centerSqrtPriceX96` to calculate new tick ranges. An attacker can manipulate the pool price, causing the protocol to rebalance its concentrated liquidity position to a range centered on the manipulated price. When the attacker restores the price, the protocol's position is out of range and earns no fees.
- `convertToV3()` (line 689): Uses `centerSqrtPriceX96` to calculate tick ranges for the new `V3` position. Manipulation causes the protocol to create its initial position at the wrong price range, permanently locking liquidity in an unfavorable range.
- `decreaseLiquidity()` (line 893): The price check is supposed to prevent removal of liquidity during manipulation (which affects the token ratio received). Without it, an attacker can manipulate the price to extract a more favorable token ratio from the protocol's position.
- `increaseLiquidity()` (line 974): The price check is supposed to prevent adding liquidity at a manipulated price (which affects the token ratio deposited). Without it, the protocol deposits at an unfavorable ratio.
- `collectFees()` (line 822): The price check prevents fee collection during manipulation (which could affect swap calculations in the same transaction).

These are **protocol-owned liquidity management operations** — they control the protocol's own concentrated liquidity positions on Uniswap `V3`. The oracle guard is their sole defense against flash-loan manipulation. The impact is Medium because exploitation requires a flash-loan manipulation of a `V3` pool in the same block as a protocol operation, and the loss magnitude depends on the pool's liquidity depth and the size of the affected operation.

Proof of Concept

[View detailed Proof of Concept](#)

Recommended Mitigation

Two fixes are needed -- one for each root cause:

Root Cause A: Revert when TWAP is unavailable instead of silently returning unprotected spot price:

```
// Line 1104-1106: REVERT instead of early return
if (oldestTimestamp + SECONDS_AGO < block.timestamp) {
    revert InsufficientObservationHistory(pool, oldestTimestamp,
SECONDS_AGO);
}

// Line 1113-1115: REVERT instead of early return
```

```

if (!success) {
    revert TwapCalculationFailed(pool);
}

```

Root Cause B: Preserve the original spot price before the overwrite:

```

// After line 1094, before the TWAP fetch:
uint160 spotSqrtPriceX96 = centerSqrtPriceX96; // Preserve spot price

// Line 1119 still overwrites centerSqrtPriceX96 (needed for return
value):
(twapPrice, centerSqrtPriceX96) = abi.decode(returnData, (uint256,
uint160));

// Line 1123: Use the ORIGINAL spot price, not the overwritten TWAP
value:
uint256 instantPrice = mulDiv(uint256(spotSqrtPriceX96),
uint256(spotSqrtPriceX96), (1 << 64));

```

[H-03] Balancer oracle uses vault balances as price and can be steered by anyone

Submitted by [Silvermist](#), also found by [Ace2](#), [AzizHack](#), [Funen](#), [holtzxx](#), [joker158](#), [LittleJoe](#), [Nerca](#), [nisedo](#), [santipu](#), [Silverwind](#), [udaykiranpedda](#), and [Yifan](#)

autonolas-tokenomics/.../contracts/oracles/BalancerPriceOracle.sol [#L72-L80](#)

BalancerPriceOracle.getPrice() does not read a Balancer oracle accumulator. It reads the pool balances directly from the Vault and returns a simple reserve ratio. That value is a spot price and it can change whenever the pool is traded.

```

(, uint256[] memory balances, ) =
IVault(balancerVault).getPoolTokens(balancerPoolId);
uint256 balanceIn = balances[direction];
uint256 balanceOut = balances[(direction + 1) % 2];
return (balanceOut * 1e18) / balanceIn;

```

updatePrice() is permissionless and writes the oracle snapshot based on the current spot price returned by getPrice(). This means any address can decide when the snapshot should be updated, and the snapshot is always derived from the spot regime that exists at that moment.

```
uint256 currentPrice = getPrice();
snapshot.cumulativePrice += snapshot.averagePrice * elapsedTime;
uint256 averagePrice = (snapshot.cumulativePrice + (currentPrice *
elapsedTime)) /
    ((snapshot.cumulativePrice / snapshot.averagePrice) + elapsedTime);
snapshot.averagePrice = averagePrice;
snapshot.lastUpdated = block.timestamp;
```

validatePrice(slippage) then compares the current spot price (getPrice()) to a band around the computed “timeWeightedAverage”. In practice, this check is only enforcing that current spot is near the oracle’s stored snapshot state. If an attacker can keep the pool in a manipulated spot regime and call updatePrice() over time, the stored snapshot can be moved away from the original market price and toward the attacker-controlled regime. Then validatePrice() can still return true while the price is far from the original reference.

Recommended mitigation steps

Do not use raw Vault balances as an oracle for slippage protection of large protocol swaps.

Proof of Concept

[View detailed Proof of Concept](#)

<https://github.dev/code-423n4/reports/blob/main/olas/2026-01-olas.md>

[H-04] Incorrect TWAP calculation in BalancerPriceOracle allows price manipulation and breaks oracle guarantees

Submitted by [BenRai](#), also found by [OxDemon](#), [Ace2](#), [Gakarot](#), [jkk812812](#), [JoshuaM33](#), [Morraez](#), [rage](#), and [santipu](#)

autonolas-tokenomics/./contracts/oracles/BalancerPriceOracle.sol [#L84-L128](#)

The BalancerPriceOracle.updatePrice() function implements an incorrect Time-Weighted Average Price (TWAP) formula that does not properly accumulate price over time, making the oracle unreliable and susceptible to manipulation.

Finding Description

The protocol implements a price oracle that attempts to provide a Time-Weighted Average Price (TWAP) from Balancer pool prices. The updatePrice() function is responsible for updating the price snapshot and calculating the average price over time, which is then used to validate new price updates through a slippage check.

The core issue lies in the TWAP calculation formula within the updatePrice() function:

```
// Update cumulative price with the previous average over the elapsed
time
snapshot.cumulativePrice += snapshot.averagePrice * elapsedTime;

// Update the average price to reflect the current price
uint256 averagePrice = (snapshot.cumulativePrice + (currentPrice *
elapsedTime)) /
    ((snapshot.cumulativePrice / snapshot.averagePrice) + elapsedTime);
```

This implementation breaks the fundamental TWAP formula in multiple ways:

Issue 1: Accumulating averagePrice instead of lastPrice

The `snapshot.cumulativePrice` price should accumulate the actual observed price over each time period. Instead,

```
// Update cumulative price with the previous average over the elapsed
time
snapshot.cumulativePrice += snapshot.averagePrice * elapsedTime;
```

the code accumulates `snapshot.averagePrice`, which is itself a derived average. This creates a recursive averaging effect where:

- First update: accumulates some initial price
- Second update: accumulates the average of the first update (not the actual price that was valid during that period)
- Third update: accumulates an average of averages

This fundamentally breaks the TWAP property because it doesn't weight each actual price observation by its duration.

Issue 2: Incorrect average calculation

A proper TWAP formula is:

$$\text{TWAP} = \Sigma(\text{price}_i \times \text{duration}_i) / \Sigma(\text{duration}_i)$$

The implemented formula attempts to recover the total elapsed time by dividing `snapshot.cumulativePrice / snapshot.averagePrice`, which:

- Loses precision through integer division
- Assumes the relationship `cumulativePrice = averagePrice × totalTime` holds, which it doesn't after the first update due to Issue 1
- Becomes increasingly inaccurate over time as rounding errors compound

Impact

This broken TWAP implementation has severe consequences:

1. **Oracle Manipulation:** The slippage check validates `currentPrice` against the calculated `averagePrice`. Since the average doesn't reflect true time-weighted prices, an attacker can:
 - Manipulate the Balancer pool price after the `minUpdateTimePeriod`
 - Call `updatePrice()` with the manipulated price
 - The broken TWAP formula will not properly account for the manipulation duration
 - Subsequent price updates will use this corrupted average, allowing further manipulation

Recommendation

Implement a proper TWAP calculation that accumulates actual prices weighted by time. For this you can use the observation pattern similar to UniswapV3 where observations (`timestamp`, `priceCumulative`) are saved in a circular buffer. Let a keeper call `updatePrice` on a regular basis to ensure continuous data. Also consider implementing a TWAP time range (e.g. 15 minutes) to ensure a proper time frame for the result.

The update function and the `getPrice` functions could look like this:

```
function update() external {
    uint256 timeElapsed = block.timestamp - lastUpdateTimestamp;

    if (timeElapsed > 0) {
        // Get spot price from Balancer pool
        uint256 spotPrice = getBalancerSpotPrice();

        // Update accumulator
        priceAccumulator += spotPrice * timeElapsed;
        lastUpdateTimestamp = block.timestamp;

        // Store observation
        observations.push(Observation({
            timestamp: block.timestamp,
            priceCumulative: priceAccumulator
        }));
    }
}

function getTWAP() public view returns (uint256) {
    uint256 currentTime = block.timestamp;
    uint256 targetTime = currentTime - PERIOD;
}
```

```

// Find observation closest to targetTime
Observation memory oldObservation = findObservation(targetTime);

uint256 priceDelta = priceAccumulator -
oldObservation.priceCumulative;
uint256 timeDelta = currentTime - oldObservation.timestamp;

return priceDelta / timeDelta;
}

```

[H-05] Insolvency via Cross-Service Reentrancy in StakingBase. withdraw

Submitted by [fullstop](#), also found by [Oxbrett8571](#), [Oxnija](#), [Iultimat3](#), [Agontuk](#), [BlueSheep](#), [brandon](#), [bransdotcom](#), [chuvak](#), [coinsspor](#), [Cryptor](#), [demonhat](#), [Destiny_rs](#), [Diavolo](#), [Drothon](#), [edoscoba](#), [eightzerofour](#), [hexcoded0033](#), [hiram](#), [ht111111](#), [ifex445](#), [kind0dev](#), [Konzy](#), [LMLLayersystem](#), [noxpenguin](#), [psyone](#), [recr4sh](#), [rookishere](#), [shalevhvs](#), [shibu0x](#), [SpicyMeatball](#), [Tupaia](#), [Yifan](#), and [yonko](#)

autonolas-registries/./contracts/staking/StakingBase.sol [#L929-L948](#)

Finding description and impact

The `_withdraw` function in `StakingBase.sol` violates the **Checks-Effects-Interactions (CEI)** pattern. It caches the global balance into a local variable `updatedBalance`, performs external calls (transfers) via a loop, and only updates the global balance state variable at the very end of the function.

Root Cause:

1. **State Update Lag:** The global balance is updated *after* external calls return.
2. **Lack of Reentrancy Guard:** The contract lacks a global `nonReentrant` modifier, allowing cross-service reentrancy even if single-service reentrancy is blocked by state deletion.

Attack Scenario:

An attacker controls two service IDs (Service A and Service B).

1. Attacker calls `unstake(Service A)`.
2. `_withdraw` reads balance (e.g., 100 ETH) into `updatedBalance`.
3. The contract sends rewards for Service A.
4. The attacker intercepts control in the `receive()` fallback and calls `unstake(Service B)`.

5. **Reentrancy:** The inner `unstake(Service B)` execution reads the *stale* global balance (still 100 ETH) because the outer execution has not yet updated it.
6. The inner execution calculates the new balance (e.g., 90 ETH) and writes it to storage.
7. The inner execution finishes. Control returns to the outer `unstake(Service A)`.
8. **State Overwrite:** The outer execution continues using its local `updatedBalance` (calculated based on the initial 100 ETH). It overwrites the global balance (e.g., setting it to 90 ETH), effectively erasing the deduction made by Service B.

Impact:

The contract's recorded balance becomes higher than its actual physical ETH balance (Insolvency). This accounting discrepancy accumulates with every attack.

Recommended mitigation steps

1. **Add Reentrancy Guard:** Inherit from `ReentrancyGuard` and apply the `nonReentrant` modifier to all external state-changing functions (`stake`, `unstake`, `claim`, `checkpointAndClaim`, `forcedUnstake`).
2. **Fix CEI Pattern:** Refactor `_withdraw` to update the global state *before* performing external transfers.

Proof of Concept

[View detailed Proof of Concept](#)

[H-06] Service owner can steal protocol tokens by exploiting reentrancy in create

Submitted by [SpicyMeatball](#), also found by [Cecuro](#) and [Race](#)

- `autonolas-registries/./contracts/ServiceManager.sol` [#L168](#)
- `autonolas-registries/./contracts/ServiceRegistry.sol` [#L270](#)

`ServiceManager.create()` does not follow the Checks-Effects-Interactions (CEI) pattern. This allows a malicious service creator to reenter the contract and modify service parameters before `ServiceRegistryTokenUtility.createWithToken()` is executed:

```
function create(  
    address serviceOwner,  
    address token,  
    bytes32 configHash,  
    uint32[] memory agentIds,  
    IService.AgentParams[] memory agentParams,  
    uint32 threshold  
) external returns (uint256 serviceId) {
```



```

        --- SNIP ---
    } else {
        --- SNIP ---
        // Call the original ServiceRegistry contract function
    >>>     serviceId = IService(serviceRegistry).create(serviceOwner,
configHash, agentIds, agentParams, threshold);
        // Create a token-related record for the service
    >>>     IServiceTokenUtility(serviceRegistryTokenUtility).createWithToken(service
Id, token, agentIds, bonds);
    }
}

```

Attack Scenario

- The attacker creates a service with a single agent and specifies a large USDC bond (e.g. ServiceRegistryTokenUtility balance);
- ServiceRegistry.create() mints a service NFT to the attacker using _safeMint;
- During the onERC721Received callback, the attacker reenters ServiceManager by calling update() and changes the bond amount to 1 wei;
- The attacker then calls ServiceManager.activateRegistration() and only pays the reduced bond amount;
- Execution resumes after the callback, and ServiceRegistryTokenUtility.createWithToken() is called using the stale bond configuration captured before reentrancy;
- Finally, the attacker terminates the service and withdraws the originally specified USDC bond from ServiceRegistryTokenUtility, effectively draining its funds.

Recommended mitigation steps

To enforce a secure CEI pattern and prevent reentrancy:

- Since ServiceRegistry.totalSupply() is known prior to minting, derive the expected serviceId in advance and call serviceRegistryTokenUtility.createWithToken() before invoking ServiceRegistry.create(). This ensures all state related to token bonds is finalized before any external calls occur.

Additionally or alternatively:

- Introduce a reentrancy guard on ServiceManager.create() and ServiceManager.update().

Proof of Concept

[View detailed Proof of Concept](#)

[H-07] Missing deadline parameter in register signatures

Submitted by [santipu](#), also found by [Dinesh11G](#) and [SpicyMeatball](#)

autonolas-registries/.../contracts/ServiceManager.sol [#L421](#)

The registerAgentsWithSignature function in the ServiceManager contract allows a service owner to register agent instances on behalf of an operator using a signed message.

```
function registerAgentsWithSignature(
    address operator,
    uint256 serviceId,
    address[] memory agentInstances,
    uint32[] memory agentIds,
    bytes memory signature
) external payable returns (bool success) {
    // Check the service owner
    address serviceOwner =
IERC721(serviceRegistry).ownerOf(serviceId);
    if (msg.sender != serviceOwner) {
        revert OwnerOnly(msg.sender, serviceOwner);
    }

    // Get the (operator | serviceId) nonce for the registerAgents
message
    // Push a pair of key defining variables into one key. Service Id
or operator are not enough by themselves
    // as another service might use the operator address at the same
time frame
    // operator occupies first 160 bits
    uint256 operatorService = uint256(uint160(operator));
    // serviceId occupies next 32 bits as serviceId is limited by the
2^32 - 1 value
    operatorService |= serviceId << 160;
    uint256 nonce = mapOperatorRegisterAgentsNonces[operatorService];
    // Get register agents message hash
    bytes32 msgHash = getRegisterAgentsHash(operator, serviceOwner,
serviceId, agentInstances, agentIds, nonce);

    // Verify the signed hash against the operator address
    _verifySignedHash(operator, msgHash, signature);

    // ...
}
```

The signature is computed over the following parameters:

- operator address
- service owner address
- service ID
- agent instance addresses
- agent IDs
- nonce

However, the message hash omits a critical parameter: a **deadline**. Without a deadline, the operator's signature remains valid indefinitely, unless invalidated by a nonce change. This allows the service owner to store a valid signature and use it arbitrarily far into the future.

For example:

1. Bob creates a service and requests Alice to register agents using her signature.
2. Alice signs the message, allowing Bob to register on her behalf.
3. For whatever reason, Bob does ends up not using the signature. Instead, Alice registers the agents directly.
4. Some time later, Alice unbonds from Bob and retrieves her deposit.
5. Bob reconfigures the service and opens registrations again.
6. Bob reuses Alice's old signature to register her agents without consent.
7. This triggers a bond transfer from Alice's account, effectively locking her funds again.

This behavior is possible because `registerAgentsWithSignature` eventually calls `registerAgentsTokenDeposit` with the operator's address:

```
// Record the actual ERC20 bond
bool isTokenSecured =

IServiceTokenUtility(serviceRegistryTokenUtility).registerAgentsTokenDeposit(
operator, serviceId, agentIds);
```

And within `registerAgentsTokenDeposit`, the contract transfers tokens directly from the operator:

```
safeTransferFrom(token, operator, address(this), totalBond);
```

As long as the operator has an active allowance for the `ServiceRegistryTokenUtility` contract, the bond can be forcibly transferred. Operators commonly maintain infinite allowances to save on gas when interacting with multiple services, increasing the likelihood of unintended bond transfers.

An operator's valid signature can be reused at any point in the future, enabling a service owner to forcibly register agent instances and trigger bond transfers from the operator's

account without current consent. This undermines operator autonomy and can lead to unexpected token lockups. Because the token transfer depends only on a previously issued signature and an existing token allowance, malicious or careless reuse can silently lock operator funds in services they no longer intend to participate in.

Recommended mitigation steps

Introduce a deadline parameter to the signed message and include it in the hash computation. Then, enforce that the deadline has not passed before accepting the signature during execution.

[H-08] Critical Logic Inversion in Price Guard Allows Flash-Loan Manipulation of Liquidity Operations

Submitted by [prazzix86](#), also found by [ayazwx](#), [jkk812812](#), [M4v3r1ck](#), and [Nyx](#)

`autonolas-tokenomics/.../contracts/pol/LiquidityManagerCore.sol` [#L1104-L1116](#)

The `checkPoolAndGetCenterPrice` function in `LiquidityManagerCore.sol` is intended to be a security gate that prevents the protocol from interacting with Uniswap V3 pools at manipulated "instant" prices. It is designed to verify the `slot0` price against a 30-minute Time-Weighted Average Price (TWAP).

However, two critical flaws in the implementation render this protection completely ineffective:

1. Logic Inversion in History Check (Mature Pools) The contract uses the following check to decide whether to bypass the TWAP verification:

```
1104: if (oldestTimestamp + SECONDS_AGO < block.timestamp) {
1105:     return centerSqrtPriceX96;
1106: }
```

In Uniswap V3, `oldestTimestamp` is the timestamp of the earliest recorded price point in the observation buffer.

If the pool is mature (has ≥ 30 mins of history), the condition `oldestTimestamp + 1800 < block.timestamp` is TRUE. **The Result:** The function returns the untrusted `slot0` price immediately, bypassing the TWAP check for exactly the pools that have sufficient data to be checked.

2. Fail-Open Architecture on Verification Failure (New Pools) If the pool is new (history < 30 mins), the code proceeds to call `getTwapFromOracle`. This call will typically revert because the Uniswap observe function requires data reaching back the full `SECONDS_AGO` period.

```

1111: (bool success, bytes memory returnData) =
address(this).staticcall(payload);
1112:
1113: // If the call has failed - observe was not successful...
1114: if (!success) {
1115:     return centerSqrtPriceX96;
1116: }

```

Instead of reverting (Fail-Closed), the protocol **catches the error and returns the untrusted price**. This creates a trivial bypass: an attacker can create a new pool, manipulate its price, and the protocol will use that manipulated price because the "security check" failed to execute.

Step-by-Step Exploit Scenario

- **Preparation:** Attacker identifies a liquidity operation (e.g., ConvertedToV3).
- **Manipulation:** Attacker creates a fresh Uniswap V3 pool for OLAS / SHITCOIN.
- **Flash Loan:** Attacker uses a flash loan to buy OLAS in this new pool, pushing the slot0 price to 100x the fair market value.
- **Protocol Trigger:** Attacker triggers the protocol to provide liquidity to this pool.

Bypass:

- checkPoolAndGetCenterPrice is called.
- The pool is new, so it tries getTwapFromOracle. observe([1800, 0]) reverts because the pool is only seconds old.
- The protocol catches the revert and returns the 100x manipulated slot0 price. **Profit:** The protocol mints a position or converts tokens based on the 100x price, allowing the attacker to effectively drain OLAS from the protocol in exchange for worthless tokens.

Impact

This is a Critical severity issue as it breaks the primary security assumption for price safety in the Protocol-Owned Liquidity (POL) module. It allows for the total drainage of assets committed to V3 liquidity operations via standard flash-loan price manipulation.

Recommended mitigation steps

The protocol should adopt a Fail-Closed security model and correct the history logic:

```

-         if (oldestTimestamp + SECONDS_AGO < block.timestamp) {
-             return centerSqrtPriceX96;
-         }
+         // Ensure the pool is mature enough to have TWAP data

```

```

+         if (oldestTimestamp + SECONDS_AGO > block.timestamp) {
+             revert PoolTooNew();
+         }

        uint256 twapPrice;
        bytes memory payload = abi.encodeCall(this.getTwapFromOracle,
(pool));
        (bool success, bytes memory returnData) =
address(this).staticcall(payload);

-         if (!success) {
-             return centerSqrtPriceX96;
-         }
+         require(success, "TWAP check failed");

```

Proof of Concept

[View detailed Proof of Concept](#)

[H-09] Missing maximum bond signature parameter

Submitted by [santipu](#), also found by [aestheticbhai](#), [Evo](#), [fullstop](#), [SiderSoner](#), [SpicyMeatball](#), and [Tupaia](#)

autonolas-registries/./contracts/ServiceManager.sol#L421

The `registerAgentsWithSignature` function in the `ServiceManager` contract allows a service owner to register agent instances on behalf of an operator using a signed message.

The signature is computed over the following parameters:

- operator address
- service owner address
- service ID
- agent instance addresses
- agent IDs
- nonce

However, this message does not include any upper bound on the bond amount. As a result, if the service configuration changes after the operator has signed the message (specifically, if the bond requirement increases) the operator may unintentionally lock significantly more funds than originally intended.

Example scenario:

1. Bob creates a service requiring `Agent1` instances, each with a bond of `1e18`.

2. Alice agrees to participate and signs a message authorizing Bob to register her agent instances.
3. Bob then updates the service and increases the bond to 100e18 per agent. The exact path for Bob is to first terminate the service before any register occurs, then update the service, and activate registrations again.
4. Bob uses Alice's original signature to register her agents with the updated bond amounts, locking 100x more funds than she intended.
5. Alice's excess funds remain locked and unavailable until the service is terminated.

This issue occurs because the signed message lacks a reference to the bond amounts, giving the service owner unilateral control over the financial implications of a previously signed authorization.

The absence of a maximum bond parameter in the signed message enables a service owner to increase the required bond after a signature has been issued, then reuse the original signature to register the operator's agents. This can lead to operators having large amounts of tokens bonded without their consent. These funds remain locked until the service is terminated, which is decided solely by the service's owner.

Recommended mitigation steps

Include a new parameter in the signed message that defines the maximum acceptable bond per agent instance. This value should be used in both the signature hash and enforced during execution to prevent unintended over-bonding.

[\[H-10\] Token Callback Reentrancy](#)

Found by [v12](#)

- [autonolas-registries/./contracts/staking/StakingBase.sol#L540](#)
- [autonolas-registries/./contracts/staking/StakingBase.sol#L929](#)
- [autonolas-registries/./contracts/ServiceManager.sol#L126](#)

Checks-Effects-Interactions violation in StakingBase: virtual `_transfer` allows reentrancy and balance desynchronization

Targets

- `_claim` (StakingBase)
- `_withdraw` (StakingBase)
- `_claim/_withdraw` (StakingBase)

Description

The StakingBase contract violates the checks-effects-interactions (CEI) pattern in its claim/withdraw flow by performing external interactions through an overridable internal

function `_transfer(...)` before persisting updated balances to storage. `_claim` zeroes per-service rewards and calls internal `_withdraw`, which computes a local `updatedBalance` and issues transfers by calling the internal virtual `_transfer` for each receiver. Because `_transfer` is abstract/virtual and the implementation may perform external calls or invoke token callbacks, a malicious receiver or token can re-enter the contract during these transfers. The contract only writes the final `updatedBalance` to storage after all transfers complete, so any reentrant withdrawal occurring during the external call window will observe stale stored balances and can pass checks multiple times. When control returns to the outer `_withdraw`, it overwrites storage with a stale balance derived before the reentrant deduction, desynchronizing stored accounting from actual token holdings and enabling theft.

Root cause

Violation of checks-effects-interactions plus unsafe use of an overridable/external-facing transfer hook. The contract computes and uses a local `updatedBalance` while performing potentially reentrant external interactions via an internal virtual `_transfer`, and only writes the persistent balance after those interactions complete. There is no reentrancy guard and the abstract/virtual `_transfer` can trigger external callbacks (e.g., token hooks, receiver fallback), enabling reentrant entry into claim/withdraw logic.

Impact

High — An attacker who can act as a payment receiver or controls a malicious token contract can reenter claim/withdraw during `_transfer` and cause overlapping withdrawals that individually pass checks but cumulatively drain more tokens than the contract's stored balance should allow. This can result in: complete or partial draining of contract-held funds, corruption/desynchronization of internal accounting, and loss of user funds. The attack requires the attacker to be a recipient (or control a recipient/token hook) and to have other claimable rewards to trigger reentrancy, but these are realistic in staking/reward systems.

Unsafe external call ordering in `ServiceManager.create` allows reentrant token-utility interaction via `onERC721Received`

Targets

- `create (ServiceManager)`

Description

`ServiceManager.create` performs an external call to `serviceRegistry.create` (which may safe-mint an ERC721) before completing or protecting subsequent cross-contract work (calling the token-utility). The safe-mint can invoke the recipient's `onERC721Received` callback while `serviceRegistry.create` is still executing. Because `ServiceManager` does not persist state or use a reentrancy guard around these interactions, a malicious recipient can run arbitrary code during the callback and invoke `token-utility.createWithToken` or re-enter `ServiceManager`, causing ordering races and inconsistent state.

Root cause

Improper ordering of external interactions and lack of reentrancy protection. The contract calls an untrusted external contract (`serviceRegistry.create`) that can synchronously call back into user-controlled code (via `ERC721Receiver` hook) before the manager completes or secures the subsequent critical operation (`createWithToken`). No persistent state updates or a reentrancy guard are used to prevent reentrant or out-of-order calls to the token-utility or manager functions.

Impact

High (context-dependent). An attacker who controls the recipient/`serviceOwner` (or otherwise triggers code during the registry's `safeMint`) can: (1) call `token-utility.createWithToken` prematurely or with manipulated inputs, (2) re-enter `ServiceManager` to create ordering races (leading to double registrations, skipped initializations, or corrupted invariants), or (3) produce inconsistent token/registry state. The exploitability and severity depend on access controls in the token-utility: if `createWithToken` is publicly callable, the issue is directly exploitable (high); if restricted, the impact may be lower but ordering/invariant violations remain possible.

[H-1] [cumulativePrice is corrupted when price updates are rejected](#)

Submitted by [santipu](#), also found by [coinsspor](#), [rox_k](#), [Silvermist](#), [StrangerMontana](#), [sunnyshrma](#), and [taronsung](#)

`autonolas-tokenomics/./contracts/oracles/BalancerPriceOracle.sol` [#L109](#)

The `BalancerPriceOracle` contract updates `cumulativePrice` before checking whether the price deviation exceeds `maxSlippage`. When the slippage check fails and the update is rejected, the function returns `false` but the `cumulativePrice` has already been permanently modified. This corrupts the TWAP calculation, causing it to diverge from the expected value.

The root cause is the ordering of operations in `updatePrice()`:

```
function updatePrice() public returns (bool) {
    // ...

    // Update cumulative price with the previous average over the
    elapsed time
    >>    snapshot.cumulativePrice += snapshot.averagePrice * elapsedTime;

    // Update the average price to reflect the current price
    uint256 averagePrice = (snapshot.cumulativePrice + (currentPrice
    * elapsedTime)) /
        ((snapshot.cumulativePrice / snapshot.averagePrice) +
    elapsedTime);
```

```

        // Check if price deviation is too high
        if (currentPrice < averagePrice - (averagePrice * maxSlippage /
100) ||
            currentPrice > averagePrice + (averagePrice * maxSlippage /
100))
        {
>>            return false;
        }

        // ...
    }

```

When the function returns false:

- `snapshot.cumulativePrice` has been increased
- `snapshot.averagePrice` remains unchanged (old value)
- `snapshot.lastUpdated` remains unchanged (old timestamp)

Example Scenario

1. Initial state:

- `cumulativePrice` = 10,000
- `averagePrice` = 100
- `lastUpdated` = T_0

2. After 100 seconds, someone calls `updatePrice()` with a price that exceeds slippage:

- `elapsedTime` = 100
- `currentPrice` = 200 (exceeds `maxSlippage`, let's say 10%)
- `cumulativePrice` = $10,000 + (100 \times 100) = 20,000$ X (modified!)
- Slippage check fails → returns false
- `averagePrice` stays at 100
- `lastUpdated` stays at T_0

3. After another 100 seconds, legitimate update at price 105:

- `elapsedTime` = 200 (time since T_0 , not 100!)
- `cumulativePrice` = $20,000 + (100 \times 200) = 40,000$

The problem: The `cumulativePrice` is now 40,000, but it should only be 30,000 if the rejected update hadn't corrupted it:

- Correct: $10,000 + (100 \times 200) = 30,000$
- Actual: $20,000 + (100 \times 200) = 40,000$

The cumulative price has an extra 10,000 from the rejected update. This error persists and affects all future TWAP calculations.

Impact

The corrupted `cumulativePrice` causes `validatePrice()` to compute an incorrect TWAP:

1. Denial of Service: The skewed TWAP may fall outside the acceptable slippage range from the current spot price, causing `validatePrice()` to incorrectly return false for legitimate operations.
2. Sandwich Attacks: When the corrupted TWAP happens to align with a manipulated spot price, attackers can exploit `BuyBackBurner` and `LiquidityManagerOptimism` by sandwiching transactions that should have been rejected.

The severity compounds over time as multiple rejected updates each add erroneous values to `cumulativePrice`.

Recommended mitigation steps

Move the slippage check before updating `cumulativePrice`, or use a local variable for the calculation and only persist to storage after validation passes.

Medium Risk Findings (12)

[M-01] Arbitrum Retryable-Ticket Refund/Value Not Verified Enables Timelock ETH Exfiltration

Submitted by [OxbOkO](#), also found by [Oxastronatey](#), [OxJoker42](#), [Oxterrah](#), [Bale](#), [Diavolo](#), [Drothon](#), [edoscoba](#), [johnyfwesh](#), [kimchiwarrior](#), [kindOdev](#), [legat](#), [Nakamoto](#), [PlayboiEvokid](#), [ProngsDev](#), and [qwqkol](#)

- `autonolas-governance/./contracts/multisigs/GuardCM.sol` [#L189-L239](#)
- `autonolas-governance/./contracts/multisigs/bridge_verifier/ProcessBridgedDataArbitrum.sol` [#L31-L60](#)

`GuardCM` is meant to restrict what the Community Multisig (CM) can schedule through the timelock by allowing only specific `target + selector (+ chainId)` combinations. For Arbitrum-bridged actions, the current verification only checks the L2 `targetAddress` and the L2 `targetPayload` selector, while ignoring (1) the timelock value forwarded on execution and (2) critical Arbitrum retryable-ticket parameters (notably the refund recipients). As a result, CM can schedule an allowlisted L2 action but still divert timelock ETH via attacker-controlled refund addresses.

Root cause

There are two coupled gaps:

1. `GuardCM._verifySchedule()` ignores the timelock value

- In `schedule(address target, uint256 value, bytes data, ...)`, the value is decoded but discarded:
 - `GuardCM.sol` decodes `(target, value, data, ...)` but never checks/limits value.
 - Same applies to `scheduleBatch(...)` (values array is decoded but not enforced).
- <https://github.com/valory-xyz/autonolas-governance/blob/6ea160398200a7b1ca6bf916fc2cd91483bff707/contracts/multisigs/GuardCM.sol#L189-L239>
- 2. **ProcessBridgedDataArbitrum validates only L2 targetAddress + selector and ignores refund recipients / fee knobs**
- `ProcessBridgedDataArbitrum.processBridgeData()` ABI-decodes the Arbitrum retryable ticket payload and extracts only:
 - `targetAddress` (L2 destination), and
 - `targetPayload` (L2 calldata), then calls `_verifyData(targetAddress, targetPayload, chainId)`.
- It does **not** verify:
 - `excessFeeRefundAddress`
 - `callValueRefundAddress`
 - `maxSubmissionCost`, `gasLimit`, `maxFeePerGas`, `l2CallValue`, etc.
- https://github.com/valory-xyz/autonolas-governance/blob/6ea160398200a7b1ca6bf916fc2cd91483bff707/contracts/multisigs/bridge_verifier/ProcessBridgedDataArbitrum.sol#L31-L60

Attack path (step-by-step)

Assume `GuardCM` is active and installed as the Safe guard for the CM Safe, and CM can schedule timelock calls.

1. CM proposes/schedules a timelock operation calling the Arbitrum Inbox (`createRetryableTicket` or `unsafeCreateRetryableTicket`).
2. The retryable ticket is crafted such that:
 - `targetAddress` is a whitelisted L2 contract,
 - `targetPayload` starts with a whitelisted selector (so `_verifyData()` passes),
 - but `excessFeeRefundAddress` and `callValueRefundAddress` are set to attacker-controlled addresses.
3. The timelock `schedule(...)` call uses a large value (ETH) to be forwarded when executing the retryable-ticket call.
4. `GuardCM.checkTransaction()` approves the schedule because:
 - it ignores/doesn't constrain the timelock value, and

- the Arbitrum bridge verifier ignores refund recipients and only verifies L2 target+selector.
5. When the timelock executes, it forwards ETH into the Arbitrum Inbox call; under Arbitrum's retryable-ticket model, excess fees/value are refunded to the provided refund recipients. Since those recipients are attacker-controlled and unverified, timelock ETH can be diverted.

Impact(s)

- **High severity: loss of funds (timelock ETH)** via attacker-controlled refund recipients, despite the guard being active.
- Bypasses the intended security boundary of GuardCM: CM can schedule "allowlisted" L2 actions while still abusing unverified parameters to redirect L1 value.
- The amount at risk is bounded by the timelock's ETH balance and the scheduled value.

Threat model / who can exploit

- Any actor who can get a transaction executed from the CM Safe while GuardCM is active (e.g., compromised CM signers, an attacker meeting the Safe threshold).
- The README explicitly declares the DAO as trusted, but GuardCM exists specifically to *constrain* CM actions; therefore, "malicious/compromised CM" is an in-scope attacker for guard bypasses.

Cost/feasibility

- Minimum attacker cost is approximately L1 gas + whatever minimum non-refundable amount the Arbitrum Inbox requires.
- The attacker's gain scales with chosen value (up to the timelock ETH balance), not with cost.

Recommended mitigation steps

1) Enforce value constraints for schedule / scheduleBatch in GuardCM

GuardCM._verifySchedule() currently decodes schedule(address target, uint256 value, bytes data, ...) but does not validate value.

- For any bridged call (i.e., when `mapBridgeMediatorL1BridgeParams[target].verifierL2 != address(0)`), enforce:
 - `value == 0` for schedule, and
 - `all values[i] == 0` for scheduleBatch.

Rationale: the guard should restrict not only *what* is called, but also *how much ETH* can be forwarded alongside the call. Without this, allowlisted calls can still be used to redirect timelock ETH via bridge-specific refund mechanics.

2) Validate Arbitrum retryable-ticket economic parameters in

`ProcessBridgedDataArbitrum` `ProcessBridgedDataArbitrum.processBridgeData()` should decode and validate the full retryable-ticket call, not only (`targetAddress`, `targetPayload`).

At minimum, enforce:

- `excessFeeRefundAddress` is an approved address (e.g., a DAO-controlled receiver), not attacker-controlled.
- `callValueRefundAddress` is an approved address (same rationale).

Recommended additional constraints (defense-in-depth):

- Require `l2CallValue == 0` unless explicitly needed for the allowlisted action.
- Enforce sane bounds for `maxSubmissionCost`, `gasLimit`, `maxFeePerGas` (or enforce they match predetermined values for the specific action).
- Consider rejecting `unsafeCreateRetryableTicket` entirely unless there is a strong need.

Proof of Concept

[View detailed Proof of Concept](#)

[M-02] Uniswap oracle validateprice can be grieved per block via `sync()`.

Submitted by [Silvermist](#), also found by [BenRai](#), [edantes](#), [edoscoba](#), [mrdafidi](#), [Nyx](#), [Valves](#), and [yaractf](#)

`autonolas-tokenomics/.../contracts/oracles/UniswapPriceOracle.sol` [#L68-L72](#)

`validatePrice()` returns `false` if the Uniswap V2 pair's `blockTimestampLast` equals the current `block.timestamp`.

Because Uniswap V2 pairs have a permissionless `sync()` that updates the pair reserves state and sets `blockTimestampLast` to the current block timestamp, a mempool attacker can front-run a victim transaction in the same block by calling `sync()` first.

When the victim later executes in that same block, `blockTimestampLast == block.timestamp`, so `validatePrice()` returns `false`.

```
(, , uint256 blockTimestampLast) = IUniswapV2(pair).getReserves();
if (block.timestamp == blockTimestampLast) {
    return false;
}
```

This is “per-block” because the failure is tied to the block timestamp: if the victim retries in the next block (different `block.timestamp`), the check can succeed again. But the attacker can repeat the same `sync()` front-run each time the victim tries, causing consistent failures for any public-mempool execution.

The victim can succeed in a later block, but the attacker can keep repeating the same front-run `sync()` whenever the action is attempted via the public mempool.

Recommended mitigation steps

Avoid making validation fail solely because the pair was updated in the same block.

Proof of Concept

[View detailed Proof of Concept](#)

[M-03] Price cumulative last is used inverted in Uniswap Oracle

Submitted by [holtzxx](#), also found by [4lifemen](#), [Aamir](#), [Cryptor](#), [santipu](#), [testnate](#), and [Varun_05](#)

autonolas-tokenomics/.../contracts/oracles/UniswapPriceOracle.sol [#L62](#)

Currently, in `UniswapPriceOracle` the function `validatePrice()` is:

```
function validatePrice(uint256 slippage) external view returns (bool)
{
    require(slippage <= maxSlippage, "Slippage overflow");

    // Compute time-weighted average price
    // Fetch the cumulative prices from the pair
    uint256 cumulativePriceLast;
    if (direction == 0) {
        cumulativePriceLast =
        IUniswapV2(pair).price1CumulativeLast(); // @audit appears to be
        inverted, ts should be price0cumulative if direction is 0
    } else {
        cumulativePriceLast =
        IUniswapV2(pair).price0CumulativeLast();
    }

    // Fetch the reserves and the last block timestamp
    (, , uint256 blockTimestampLast) =
    IUniswapV2(pair).getReserves();

    // Require at least one block since last update
```

```

    if (block.timestamp == blockTimestampLast) {
        return false;
    }
    uint256 elapsedTime = block.timestamp - blockTimestampLast;

    uint256 tradePrice = getPrice();

    // Calculate cumulative prices
    uint256 cumulativePrice = cumulativePriceLast + (tradePrice *
elapsedTime);

    // Calculate the TWAP for OLAS in terms of native token
    uint256 timeWeightedAverage = (cumulativePrice -
cumulativePriceLast) / elapsedTime;

    // Get the final derivation to compare with slippage
    // Final derivation value must be
    uint256 derivation = (tradePrice > timeWeightedAverage)
        ? ((tradePrice - timeWeightedAverage) * 1e16) /
timeWeightedAverage
        : ((timeWeightedAverage - tradePrice) * 1e16) /
timeWeightedAverage;

    return derivation <= slippage;
}

```

The meanings of priceCumulativeLast is that, if:

price0CumulativeLast -> will return the price of token1 denominated in token0

price1CumulativeLast -> will return the price of token0 denominated in token1

In constructor, we build out the direction that will denominate who is token0 and who is token1 as follows:

```

    constructor(address _secondToken, uint256 _maxSlippage, address
_pair) {
        pair = _pair;
        maxSlippage = _maxSlippage;

        // Get token direction
        address token0 = IUniswapV2(pair).token0();
        if (token0 != _secondToken) {
            direction = 1;
        }
    }
}

```


The whole goal of the oracle is to give the price of OLAS in units of second token (This is because it will be used in BuyBack since we swap a given second token to OLAS).

If the direction is 0, which means the secondToken is the token0, then we will fetch price1CumulativeLast

```
if (direction == 0) {
    cumulativePriceLast =
    IUniswapV2(pair).price1CumulativeLast(); // @audit appears to be
    inverted, ts should be price0cumulative if direction is 0
```

Which will return the price of token0 denominated in token1 (Price of second token denominated in olas, which is exactly the inverse of what we want).

When direction is 1, which means the second token is the token1, we will fetch price0CumulativeLast, which returns the price of token1 denominated in token0 (Price of second token denominated in olas again).

You can see in BuyBackBurner how we used the validatePrice function for instance:

```
function _buyOLAS(address secondToken, uint256 secondTokenAmount)
internal virtual returns (uint256 olasAmount) {
    // Get oracle address
    address poolOracle = mapV2Oracles[secondToken];

    // Check for zero address
    require(poolOracle != address(0), "Zero oracle address");

    // Apply slippage protection
    require(IOracle(poolOracle).validatePrice(maxSlippage), "Before
    swap slippage limit is breached");

    // Get current pool price
    uint256 previousPrice = IOracle(poolOracle).getPrice();
```

The pool oracle is of the second token (Not OLAS, so we need to get the price of OLAS denominated in the second token).

Impact

If the price moves from 2 -> 1 for instance, we will compute as it was 0.5 -> 1 which is a gain of 100%, given it is the inverse.

And therefore the slippage check may pass when it shouldn't.

This will allow trading at a wrong price, so users could be charged more, or gain more depending on how the price diverges. (Buybacks in Uniswap V2 will trade at a price that is not quite the market price, it is the inverse).

Mitigation

```
if (direction == 0) {
    cumulativePriceLast =
    IUniswapV2(pair).price0CumulativeLast(); // @audit appears to be
    inverted, ts should be price0cumulative if direction is 0
} else {
    cumulativePriceLast =
    IUniswapV2(pair).price1CumulativeLast();
}
```

Proof of Concept

[View detailed Proof of Concept](#)

[M-04] DoS in Liquidity Migration due to Unit Mismatch in UniswapPriceOracle

Submitted by [ElmInNyc99](#), also found by [Oxcode](#), [Oxdonchev](#), [harry](#), [jkk812812](#), [NeuroSpire](#), [NexusAudits](#), [osok](#), and [Viveksh0062](#)

`autonolas-tokenomics/.../contracts/oracles/UniswapPriceOracle.sol` [#L86-L90](#)

The `UniswapPriceOracle.validatePrice()` function suffers from a logic error due to a unit mismatch. It calculates price derivation using `1e16` precision but compares it against a slippage parameter provided in raw percentage units. This causes the validation to fail for almost any price deviation, effectively causing a Denial of Service (DoS) for the V2-to-V3 liquidity migration in `LiquidityManagerETH`.

Root Cause

The `validatePrice()` function calculates derivation scaled to `1e16` (where `1e16` = 100%), but the input slippage is provided as a raw integer (e.g., `50` represents 50%).

```
uint256 derivation = (tradePrice > timeWeightedAverage)
    ? ((tradePrice - timeWeightedAverage) * 1e16) /
    timeWeightedAverage
    : ((timeWeightedAverage - tradePrice) * 1e16) /
    timeWeightedAverage;
```

```
return derivation <= slippage;
```

1.

- Formula: $\text{derivation} = (\text{diff} * 1e16) / \text{timeWeightedAverage}$
- Scenario: A 5% price deviation results in $\text{derivation} = 5e14$.
- Check: $5e14 \leq 50$ evaluates to FALSE.
- Consequently, any deviation greater than 0.00000005% triggers a revert.

2. **Inconsistent Implementation:** Unlike BalancerPriceOracle, which correctly normalizes these values, UniswapPriceOracle lacks the necessary scaling logic.

- All calls to `LiquidityManagerETH.convertToV3()` (which internally calls `_checkTokensAndRemoveLiquidityV2`) will revert with `SlippageLimitBreached()`.

```
// Apply slippage protection via V2 oracle: transform BPS into %
as required by the function
if (!IOracle(oracleV2).validatePrice(maxSlippage / 100)) {
    revert SlippageLimitBreached();
}
```

Impact

Users are unable to migrate liquidity from Uniswap V2 to V3, rendering the feature unusable.

Recommendation

Scale the slippage parameter to match the 1e16 precision used for derivation.

Recommended Fix:

```
function validatePrice(uint256 slippage) external view returns (bool) {
    require(slippage <= maxSlippage, "Slippage overflow");

    // ... calculate derivation ...

    uint256 derivation = (tradePrice > timeWeightedAverage)
        ? ((tradePrice - timeWeightedAverage) * 1e16) /
        timeWeightedAverage
        : ((timeWeightedAverage - tradePrice) * 1e16) /
        timeWeightedAverage;

    // FIX: Convert slippage from % to 1e16 format (multiply by 1e14)
    return derivation <= (slippage * 1e14);
}
```

```
}
```

Proof of Concept

[View detailed Proof of Concept](#)

[M-05] [BalancerPriceOracle::validatePrice](#) uses stale TWAP

Submitted by [santipu](#), also found by [LinKenji](#), [M4v3r1ck](#), and [PlayboiEvokid](#)

`autonolas-tokenomics/./contracts/oracles/BalancerPriceOracle.sol` [#L132](#)

The `BalancerPriceOracle` contract's `validatePrice()` function calculates the time-weighted average price using potentially outdated snapshot data because it never calls `updatePrice()` to refresh the oracle state before validation.

The root cause is that `validatePrice()` is a view function that only reads from storage without ensuring the underlying TWAP data has been recently updated current:

```
function validatePrice(uint256 slippage) external view returns (bool)
{
    // ...

    // Compute time-weighted average price
    >> uint256 timeWeightedAverage = (snapshot.cumulativePrice +
    (snapshot.averagePrice * elapsedTime)) /
    ((snapshot.cumulativePrice / snapshot.averagePrice) +
    elapsedTime);

    uint256 tradePrice = getPrice();

    // ...
}
```

The TWAP calculation uses `snapshot.averagePrice` and `snapshot.cumulativePrice`, which are only updated when `updatePrice()` is explicitly called. If no one has called `updatePrice()` for an extended period, the formula extrapolates from stale data:

- `snapshot.averagePrice` reflects the price from the last successful update, not the current market.
- `snapshot.cumulativePrice` hasn't accumulated price data for the intervening period.
- The `elapsedTime` grows unboundedly, causing the stale `averagePrice` to dominate the TWAP calculation.

For example, if the oracle was last updated 24 hours ago at price X, and the current spot price is Y (significantly different from X), the TWAP calculation will be heavily weighted toward the outdated price X. This produces a TWAP that doesn't accurately reflect recent market activity.

The problem is exacerbated because `updatePrice()` is a public function with no incentive mechanism—there's no guarantee anyone will call it regularly. The contracts that depend on `validatePrice()` (`BuyBackBurner` and `LiquidityManagerOptimism`) don't call `updatePrice()` before validation either:

```
function _buyOLAS(address secondToken, uint256 secondTokenAmount)
internal virtual returns (uint256 olasAmount) {
    // Get oracle address
    address poolOracle = mapV2Oracles[secondToken];

    // Check for zero address
    require(poolOracle != address(0), "Zero oracle address");

    // Apply slippage protection
    >> require(IOracle(poolOracle).validatePrice(maxSlippage), "Before
swap slippage limit is breached");

    // ...
}
```

```
function _checkTokensAndRemoveLiquidityV2(address[] memory tokens,
bytes32 v2Pool)
internal
virtual
override
returns (uint256[] memory amounts)
{
    // ...

    // Apply slippage protection via V2 oracle: transform BPS into %
as required by the function
    >> if (!IOracle(oracleV2).validatePrice(maxSlippage / 100)) {
        revert SlippageLimitBreach();
    }

    // ...
}
```

This leads to two failure modes:

1. **False negatives:** Legitimate operations are blocked because the stale TWAP doesn't reflect the current market price, causing `validatePrice()` to return false even when the spot price is fair.
2. **False positives:** If the current spot price happens to align with the stale TWAP (potentially through manipulation), `validatePrice()` returns true even though the TWAP is based on outdated data and doesn't represent actual recent market activity.

Given that `validatePrice()` is the core protection mechanism for both `BuyBackBurner` and `LiquidityManagerOptimism`:

1. **BuyBackBurner:** Buyback operations may be incorrectly blocked or incorrectly allowed based on stale price data, enabling either DoS or sandwich attacks.
2. **LiquidityManagerOptimism:** Liquidity withdrawals face the same risk—legitimate withdrawals blocked, or manipulated withdrawals allowed through.

So the impact on both contracts are either a DoS under normal conditions, or allowed price manipulation that causes a loss of funds for the protocol due to sandwich attacks.

Recommended mitigation steps

There are different approaches to mitigate this issue:

- Modify `validatePrice()` to call `updatePrice()` internally before performing the TWAP calculation. This requires changing `validatePrice()` from a view function to a state-modifying function.
- Update the functions calling `validatePrice()` so they always call `updatePrice()`.
- Update `validatePrice()` so that it returns false if the TWAP hasn't been updated in a long time.

[\[M-O6\] changeRanges silently fails when price is out of tick range, sending all liquidity to treasury instead of creating new position](#)

Submitted by [anchabadze](#), also found by [hirusha](#), [JohnLaw](#), [Nyx](#), [santipu](#), and [yonko](#)
`autonolas-tokenomics/./contracts/pol/LiquidityManagerCore.sol#L770`

The `changeRanges` function in `LiquidityManagerCore.sol` is designed to reposition an existing Uniswap V3 liquidity position to a new tick range. The function removes liquidity from the old position, collects all tokens, and creates a new position with the collected amounts.

However, when the current pool price is outside the position's tick range, Uniswap V3's `collect()` returns one of the token amounts as zero (all liquidity is concentrated in one token). The function checks `if (amounts[0] > 0 && amounts[1] > 0)` before creating a new position, but when this condition fails, the function:

1. Does NOT revert

2. Does NOT create a new position
3. Sends ALL collected tokens to treasury via `_manageUtilityAmounts`
4. Leaves `mapPoolAddressPositionIds[pool]` pointing to the old (now empty) position

This is a silent failure - the owner expects a new position but instead sending all protocol-owned liquidity to treasury.

```
// LiquidityManagerCore.sol
// Check that we have liquidity for both tokens
if (amounts[0] > 0 && amounts[1] > 0) {
    // ... create new position ...
    mapPoolAddressPositionIds[pool] = positionId;
}
// No else clause - silent failure!

// Manage token leftovers - transfer both to treasury
_manageUtilityAmounts(tokens, MAX_BPS, false); // ALL tokens go here
```

Scenario

1. Protocol has an active LP position in OLAS/WETH pool earning fees with tick range `[-10000, +10000]`.
2. Market conditions change significantly - OLAS price drops and current tick moves to `-15000`, outside the position's range.
3. Owner calls `changeRanges()` to reposition liquidity to a new range that includes current price.
4. `decreaseLiquidity()` removes all liquidity from old position.
5. `collect()` returns all tokens, but since price is out of range, one amount is 0 (e.g., `amounts = [1000000 OLAS, 0 WETH]`).
6. The condition `amounts[0] > 0 && amounts[1] > 0` evaluates to `false`, skipping new position creation.
7. `_manageUtilityAmounts()` transfers all 1000000 OLAS to treasury.
8. Function completes successfully without revert - owner has no indication of failure.
9. `mapPoolAddressPositionIds[pool]` still points to old position ID which now has zero liquidity.
10. Protocol loses active LP position and stops earning trading fees indefinitely.
11. `mapPoolAddressPositionIds` mapping points to an old (empty) position

Impact

1. All tokens from the LP position are sent to treasury instead of being redeployed to a new position.
2. Protocol stops earning trading fees from the affected pool.
3. No revert or error indication - owner believes operation succeeded.
4. `mapPoolAddressPositionIds` mapping points to an empty position. If owner subsequently calls `convertToV3` or `increaseLiquidity()` intending to add tokens to the "new" position, the tokens are instead added to the old position with the old tick range that is still out of current price range. This means newly deposited tokens also earn zero trading fees. The owner believes they are operating on a correctly positioned LP, but in reality all subsequent liquidity additions compound the loss by depositing into an out-of-range position. Additionally, `collectFees()` and `decreaseLiquidity()` will revert with `ZeroValue` error, potentially causing confusion and blocking normal protocol operations until the issue is manually diagnosed and resolved.
5. Owner must identify the problem, understand the cause, then manually recover: call `transferPositionId()` to clear the mapping, withdraw tokens from treasury back to the `LiquidityManager`, and redeploy liquidity via `convertToV3`, incurring additional gas costs and potential slippage, which affects the protocol

Recommended Mitigation

Handle the out-of-range case inside `changeRanges()` so the owner does not have to recover manually. When `amounts[0] == 0 || amounts[1] == 0` after `_collectFees()`, do not revert and do not send everything to treasury; instead, open a new position using the single token (single-sided liquidity): compute tick bounds so that the current pool price lies outside the new range, then mint the new position with that one token and update `mapPoolAddressPositionIds[pool]`. Use the existing two-token flow when both amounts are non-zero.

Optionally, if the protocol prefers to disallow single-sided behaviour here, revert with a clear error (e.g. `LiquidityOutOfRange`) so the transaction rolls back and the owner can retry when price is in range or exit via `decreaseLiquidity()` and then recover via `transferPositionId()`, treasury withdrawal, and `convertToV3()`.

Proof of Concept

[View detailed Proof of Concept](#)

[M-07] Malicious user can prevent buying back OLAS token via Slipstream

Submitted by [mrMorningstar](#), also found by [Aristos](#)

autonolas-
tokenomics/blame/bbec5ac12721a62672fb7a5ffba5c40f5a46d8cb/contracts/utils/BuyBackBurner.sol [#L70](#)

Whenever buying back olas occurs via Slipstream V3 it goes through [buyBack](#) first:

```
function buyBack(address secondToken, uint256 secondTokenAmount, int24
feeTierOrTickSpacing) external virtual {
    // Reentrancy guard
    if (_locked > 1) {
        revert ReentrancyGuard();
    }
    _locked = 2;

    // Get token balance
    uint256 balance = IERC20(secondToken).balanceOf(address(this));

    // Adjust second token amount, if needed
    if (secondTokenAmount == 0 || secondTokenAmount > balance) {
        secondTokenAmount = balance;
    }

    if (secondTokenAmount == 0) {
        revert ZeroValue();
    }

    // Record msg.sender activity
    mapAccountActivities[msg.sender]++;

    // Buy OLAS
    uint256 olasAmount = _buyOLAS(secondToken, secondTokenAmount,
feeTierOrTickSpacing);

    emit BuyBack(secondToken, secondTokenAmount, olasAmount);

    // Get OLAS contract balance
    olasAmount = IERC20(olas).balanceOf(address(this));

    // Transfer OLAS to bridge2Burner contract
    IERC20(olas).transfer(bridge2Burner, olasAmount);

    emit TokenTransferred(bridge2Burner, olasAmount);

    _locked = 1;
}
```

Then [_buyOLAS](#) is called:

```
function _buyOLAS(address secondToken, uint256 secondTokenAmount, int24
feeTierOrTickSpacing)
    internal
    virtual
    returns (uint256 olasAmount)
{
    address localOlas = olas;

    address[] memory tokens = new address[](2);
    (tokens[0], tokens[1]) = (secondToken > localOlas) ? (localOlas,
secondToken) : (secondToken, localOlas);

    // Get factory from LiquidityManager
    // Actual factoryV3 is fetched from LiquidityManager, since
    LiquidityManager is proxy and factory might change
    address factoryV3 =
    ILiquidityManager(liquidityManager).factoryV3();

    // Get V3 pool from liquidity manager
    address pool = getV3Pool(factoryV3, tokens,
feeTierOrTickSpacing);

    // Check for whitelisted pool address
    if (!mapV3Pools[pool]) {
        revert UnauthorizedPool(pool);
    }

    // Apply slippage protection

    ILiquidityManager(liquidityManager).checkPoolAndGetCenterPrice(pool);

    // Perform swap to OLAS
    olasAmount = _performSwap(secondToken, secondTokenAmount,
feeTierOrTickSpacing);
}
```

Then it will invoke [_performSwap](#) on `BuyBackBurnerBalancer.sol`:

```
/// @dev Performs swap for OLAS on Slipstream CL DEX.
/// @param secondToken Second token address.
```

```

    /// @param secondTokenAmount Second token amount.
    /// @param tickSpacing Tick spacing.
    /// @return olasAmount Obtained OLAS amount.
    function _performSwap(address secondToken, uint256 secondTokenAmount,
int24 tickSpacing)
        internal
        virtual
        override
        returns (uint256 olasAmount)
    {
        IERC20(secondToken).approve(swapRouter, secondTokenAmount);

        ISwapRouter.ExactInputSingleParams memory params =
ISwapRouter.ExactInputSingleParams({
            tokenIn: secondToken,
            tokenOut: olas,
            tickSpacing: tickSpacing,
            recipient: address(this),
            deadline: block.timestamp,
            amountIn: secondTokenAmount,
            amountOutMinimum: 1,
            sqrtPriceLimitX96: 0
        });

        // Swap tokens
        olasAmount = ISwapRouter(swapRouter).exactInputSingle(params);
    }

```

The function will try to swap secondToken for olas token via [exactInputSingle](#):

```

function exactInputSingle(ExactInputSingleParams calldata params)
    external
    payable
    override
    checkDeadline(params.deadline)
    returns (uint256 amountOut)
{
    amountOut = exactInputInternal(
        params.amountIn,
        params.recipient,
        params.sqrtPriceLimitX96,
        SwapCallbackData({
            path: abi.encodePacked(params.tokenIn,
params.tickSpacing, params.tokenOut),
            payer: msg.sender

```

```

        })
    );
    require(amountOut >= params.amountOutMinimum, "Too little
received");
    refundETH();
}

```

As we can see after swap is successfully executed the function will invoke [refundETH](#) to send back any eth that is in the contract (it is not necessary that eth is provided by the caller of the swap function):

```

function refundETH() public payable override nonReentrant {
    if (address(this).balance > 0)
        TransferHelper.safeTransferETH(msg.sender, address(this).balance);
}

```

The function uses [safeTransferETH](#) which will send all eth balance to the msg.sender (BuyBackBurnerBalancer aka BuyBackBurnerProxy in our case):

```

function safeTransferETH(address to, uint256 value) internal {
    (bool success,) = to.call{value: value}(new bytes(0));
    require(success, "STE");
}

```

So how this can be exploited ?

The exploit is very simple.

Malicious user can frontrun buyBack function and transfer some eth to the pool on Slipstream.

After protocol swap occur since eth balance would be bigger than 0 the function will try to send all available eth to our protocol and since BuyBackBurnerBalancer does not have functionality to receive eth the swap will revert with STE error. Which is proved in PoC below.

The [fallback](#) in BuyBackBurnerProxy is not enough which I will explain below and also prove in the PoC provided.

Let's go step by step:

1. safeTransferETH does to.call{value: value}(new bytes(0)) — i.e. an external call to the proxy with empty calldata and non-zero value. This is an ordinary ETH transfer + external call.

2. The EVM delivers that call to the proxy address. The proxy has a `fallback()` external payable, so it accepts the external call and starts executing its fallback.
3. Inside the proxy fallback protocol do an assembly delegatecall into the implementation, forwarding the same calldata (empty) and the same `msg.value`.
4. Because the incoming calldata is empty, the implementation's fallback/receive handler is what gets executed by the delegatecall.
5. Since implementation does not define a `receive()` or a payable `fallback()`, so the Solidity-generated behavior for an empty-calldata call with non-zero `msg.value` is to revert. Thus the delegatecall fails (returns false).
6. The proxy fallback sees the delegatecall failed and therefore reverts.
7. Because the proxy reverted, the original `to.call{value:...}` in `safeTransferETH` returns `success == false`, and `require(success, "STE")` triggers a revert in `safeTransferETH`. The transfer fails and no ETH is credited anywhere.

The malicious user can just call `refundETH` since it is public to get back his ETH after successfully prevented swap and therefore buying back olas token for the protocol so the attack will be at no significant cost to him and he can keep that repeating.

Impact:

Buying back olas token and transferring it to Bridge2Burner will be impossible and DoS-ed.

Recommended mitigation steps

In order to prevent this type of attack add `receive` or `fallback` functions to the implementation.

Proof of Concept

[View detailed Proof of Concept](#)

[M-08] Incorrect proportional reward splits when an operator has been slashed

Submitted by [santipu](#), also found by [rzizah](#)

autonolas-registries/./contracts/staking/StakingBase.sol [#L669-L693](#)

The `StakingBase` contract allows service owners to select a reward distribution type when allocating rewards. The available types are:

```
enum RewardDistributionType {
    // Rewards are divided as per where stake comes from,
    proportional
    Proportional,
    // Rewards go to service owner
```

```

    ServiceOwner,
    // Rewards go to service multisig
    ServiceMultisig,
    // Custom rewards distribution
    Custom
}

```

The function `_getRewardReceiversAndAmounts` determines reward allocations based on the selected distribution type. For the `Proportional` type:

```

    if (rewardDistributionType == RewardDistributionType.Proportional) {
        // Get service agent instances
        (uint256 numInstances, address[] memory agentInstances) =
IService(serviceRegistry).getAgentInstances(serviceId);

        // Total number of receivers: 1 (serviceOwner) + numInstances
(number of operators)
        uint256 totalNumReceivers = numInstances + 1;

        // Allocate arrays
        receivers = new address[](totalNumReceivers);
        amounts = new uint256[](totalNumReceivers);

        // Default setup implies that all bonds are equal
        // Get each operator reward
        uint256 operatorReward = reward / (totalNumReceivers);

        // Get corresponding operators and set operators reward amounts
        for (uint256 i = 0; i < numInstances; ++i) {
            receivers[i] =
IService(serviceRegistry).mapAgentInstanceOperators(agentInstances[i]);
            amounts[i] = operatorReward;
        }

        // Set service owner address and its reward amount
        receivers[numInstances] = serviceOwner;
        // Service owner gets its reward amount and a division remainder,
if any
        amounts[numInstances] = reward - (numInstances * operatorReward);
    }

```

As noted in the comments, this logic assumes all operator bonds are equal. Each operator is assigned an equal portion of the reward, under the assumption that each has deposited the same bond amount. However, this approach fails when an operator has been slashed.

Example scenario:

1. A service has three instances, managed by Bob, Alice, and Charlie.
2. Each operator has deposited 100 OLAS as bond, entitling each to 25% of the rewards (with the remaining 25% going to the service owner).
3. Bob's instance malfunctions and is fully slashed by Alice and Charlie, reducing his bond to zero (via `ServiceRegistry::slash`).
4. Despite this, `StakingBase` is unaware of the slashing and continues allocating 25% of the rewards to Bob.
5. Bob now earns rewards without contributing any stake or activity, diluting the rewards meant for the active, legitimate operators.

As illustrated, the proportional distribution mechanism becomes inaccurate as soon as one or more operators are slashed. Slashed operators continue to receive rewards, reducing the effective share of active participants.

The only mitigation in the current design is for the service owner to unstake and restake the service, which may be blocked by `minStakingDuration`. Even if unstaking is possible, doing so mid-period would forfeit part of the earned rewards, making the loss unavoidable.

Recommended mitigation steps

Update `_getRewardReceiversAndAmounts` so that when the distribution type is `Proportional`, the function queries the actual bond amounts for each operator. Operators with reduced or zero bonds (due to slashing) should receive proportionally less or no rewards. This ensures accurate and fair distribution.

[\[M-09\] `checkpoint\(\)` does not correct `effectiveBond` downward at year boundaries where inflation decreases](#)

Submitted by [Aleksa](#), also found by [DDooTo](#)

- [autonolas-tokenomics/./contracts/Tokenomics.sol#L1163-L1177](#)
- [autonolas-tokenomics/./contracts/Tokenomics.sol#L1278-L1280](#)
- [autonolas-tokenomics/./contracts/TokenomicsConstants.sol#L85-L96](#)

At the end of each epoch, `checkpoint()` pre-credits `effectiveBond` with the next epoch's `maxBond` (line 1279-1280):

```
curMaxBond += effectiveBond;
effectiveBond = uint96(curMaxBond);
```

This `maxBond` is computed using the current `inflationPerSecond`. When the next epoch actually settles, the code compares the **actual** `maxBond` for the settled epoch

(incentives[4], computed from the actual inflation that occurred) against the **predicted** maxBond that was pre-credited:

```
// Line 1161
incentives[4] = (inflationPerEpoch * tp.epochPoint.maxBondFraction) /
100;

// Line 1166
uint256 curMaxBond = maxBond;

// Line 1172-1177
// This has to be always true, or incentives[4] == curMaxBond if the
epoch is settled exactly at the epochLen time
if (incentives[4] > curMaxBond) {
    // Adjust the effectiveBond
    incentives[4] = effectiveBond + incentives[4] - curMaxBond;
    effectiveBond = uint96(incentives[4]);
}
```

The adjustment at line 1173 only fires when `incentives[4] > curMaxBond` — i.e., when the actual bond allocation **exceeds** the prediction. There is no `else` branch for the reverse case. The developer comment at line 1172 explicitly states this condition "has to be always true", assuming checkpoints always settle late (actual time > epochLen), so actual inflation always exceeds predicted.

This assumption breaks at year boundaries where inflation **decreases**. When an epoch crosses such a boundary, `inflationPerEpoch` is computed as a blend of the old (higher) and new (lower) rates (lines 1138-1147):

```
inflationPerEpoch = (yearEndTime - prevEpochTime) *
curInflationPerSecond;
curInflationPerSecond = getInflationForYear(numYears) / ONE_YEAR;
inflationPerEpoch += (block.timestamp - yearEndTime) *
curInflationPerSecond;
```

The blended `inflationPerEpoch` is lower than what the pre-credited `maxBond` assumed (which was based entirely on the old, higher rate). So `incentives[4] < curMaxBond`, the `if` condition is false, and the over-credit is never subtracted from `effectiveBond`. This phantom capacity persists forever.

The OLAS inflation schedule ([TokenomicsConstants.sol#L85-96](#)) has exactly two decreasing transitions:

TRANSITION	OLD INFLATION	NEW INFLATION	DECREASE
Year 2 → 3	40,400,000 OLAS	25,260,023 OLAS	-37.5%
Year 9 → 10	30,161,788 OLAS	~15,234,531 OLAS	-49.5%

After year 10, inflation is 2% compound on an increasing supply cap (always increasing), so the bug never fires again.

Impact

At each of the two inflation-decreasing year boundaries, `effectiveBond` retains phantom bond capacity equal to the difference between the pre-credited `maxBond` (computed at the old rate) and the actual blended `maxBond` for the settled epoch. The PoC below demonstrates ~**346,068 OLAS** of phantom capacity at the Year 2→3 boundary alone, with a comparable amount at Year 9→10.

This phantom capacity is real and actionable: `reserveAmountForBondProgram()` (line 803) uses `effectiveBond` as its sole gatekeeper. The Depository can create bond programs against this phantom, and when bonds mature, the Treasury mints actual OLAS to pay bondholders. The total phantom across both transitions is approximately ~**412,000 OLAS** (~0.04% of total supply), representing a violation of the inflation schedule's intended bond allocation.

The bug triggers automatically at the two year boundaries with no admin action required.

Recommended Mitigation

Add a downward correction in the `else` branch to subtract the over-credit when the actual `maxBond` is less than predicted:

```

if (incentives[4] > curMaxBond) {
    // Adjust the effectiveBond
    incentives[4] = effectiveBond + incentives[4] - curMaxBond;
    effectiveBond = uint96(incentives[4]);
+ } else if (incentives[4] < curMaxBond) {
+     // Adjust the effectiveBond downward when actual maxBond is less
+     // than pre-credited
+     effectiveBond = uint96(effectiveBond - (curMaxBond -
+ incentives[4]));
+ }

```

Proof of Concept

[\[M-10\] Services can earn undeserved rewards by manipulating checkpoint timing during reward droughts](#)

Submitted by [Takarez](#), also found by [Aristos](#), [Avalance](#), [Cecuro](#), [fullstop](#), and [Valves](#)

`autonolas-registries/./contracts/staking/StakingBase.sol` [#L635](#)

A time manipulation vulnerability allows inactive services to earn rewards for periods they provided no service. When `availableRewards` is zero, checkpoint operations are skipped, creating unmeasured time gaps that services can exploit by becoming active only when rewards become available again.

Vulnerability Details

The checkpoint mechanism in [StakingBase.sol#L635](#) only executes when rewards are available:

```
if (size > 0 && block.timestamp - tsCheckpointLast >= livenessPeriod &&
    lastAvailableRewards > 0) {
    // Activity checking and reward distribution logic
}
```

When `availableRewards == 0`, the global `tsCheckpoint` timestamp is not updated, creating time measurement gaps. Activity is measured from the last checkpoint timestamp in [StakingBase.sol#L650-665](#):

```
// Get the last service checkpoint: staking start time or the global
// checkpoint timestamp
uint256 serviceCheckpoint = tsCheckpointLast; // Uses stale timestamp
// when no rewards
uint256 ts = sInfo.tsStart;
if (ts > serviceCheckpoint) {
    serviceCheckpoint = ts;
}

// Calculate activity over the entire gap period
ts = block.timestamp - serviceCheckpoint;
bool ratioPass = _checkRatioPass(sInfo.multisig, sInfo.nonces, ts);
```

The activity check in [StakingActivityChecker.sol#L54-59](#) calculates the ratio over the entire time period:

```

if (ts > 0 && curNonces[0] > lastNonces[0]) {
    uint256 ratio = ((curNonces[0] - lastNonces[0]) * 1e18) / ts;
    ratioPass = (ratio >= livenessRatio);
}

```

This allows services to appear active by executing transactions only at the end of long inactive periods.

Impact

- Services receive compensation for periods of inactivity

Tools Used

Foundry

Recommended Mitigation

Separate activity tracking from reward distribution by maintaining activity state regardless of reward availability:

```

function _calculateStakingRewards() internal view returns (...) {
    uint256 tsCheckpointLast = tsCheckpoint;
    lastAvailableRewards = availableRewards;

    // Always track activity regardless of reward availability
    if (size > 0 && block.timestamp - tsCheckpointLast >= livenessPeriod)
    {
        // Track activity and inactivity
        _updateServiceActivity(serviceIds, tsCheckpointLast);

        // Only distribute rewards if available
        if (lastAvailableRewards > 0) {
            _calculateRewardDistribution(serviceIds,
lastAvailableRewards);
        }
    }
}

```

Proof of Concept

[View detailed Proof of Concept](#)

[\[M-11\] No slippage protection on Uniswap swap](#)

Found by [v12](#)

autonolas-tokenomics/./contracts/utils/BuyBackBurnerBalancer.sol [#L106](#)

Targets

- `_performSwap` (BuyBackBurnerBalancer)

Description

The function unconditionally sets `amountOutMinimum` to 1 when performing the Uniswap V3 swap. This allows the swap to execute even if the output amount is almost zero, leaving the contract unprotected against slippage or price manipulation.

Root cause

`amountOutMinimum` parameter in the Uniswap router call is hard-coded to 1 instead of deriving from configuration or validating the expected output.

Impact

An attacker can front-run or manipulate pool prices so that the contract receives almost no OLAS tokens in exchange for the input, leading to a loss of funds for the protocol.

[\[M-12\] Balancer oracle deadlock from cumulative price weight](#)

Submitted by [Valves](#), also found by [OxGutzzz](#), [OxVrka](#), [akleeko22](#), [DQH1](#), [Gakarot](#), [KyleMcodes](#), [M4v3r1ck](#), [sahuang](#), [santipu_](#), [udaykiranpedda](#), [viper7882](#), and [Wojack](#)

autonolas-tokenomics/./contracts/oracles/BalancerPriceOracle.sol [#L109](#)

The `BalancerPriceOracle.updatePrice()` function can enter a permanent deadlock state where price updates are perpetually rejected due to the ever-growing `cumulativePrice` **making the time-weighted average price increasingly resistant to change**. Once the cumulative weight becomes sufficiently large, any market movement exceeding `maxSlippage` will permanently brick the oracle.

Finding Description

The oracle calculates a time-weighted average price using the formula:

```
uint256 averagePrice = (snapshot.cumulativePrice + (currentPrice *
elapsedTime)) /
    ((snapshot.cumulativePrice / snapshot.averagePrice) + elapsedTime);
```

At each update, `cumulativePrice` is incremented by `snapshot.averagePrice * elapsedTime`.

The vulnerability arises from the interaction between:

1. **Growing cumulative weight:** `cumulativePrice / snapshot.averagePrice` represents TOTAL historical time weight
2. **Slippage validation:** Current price must be within `maxSlippage` of the newly calculated average

As the oracle runs longer, the denominator (`cumulativePrice / snapshot.averagePrice`) + `elapsedTime` grows dominated by the historical weight. This makes the average price increasingly inertial - even long time periods cannot shift it significantly.

Mathematical example

```
// PHASE 1: After 1 year at stable 1.0 price
elapsedTime = 31_536_000 seconds (365 days)
currentPrice = 1e18
averagePrice = 1e18

// First update succeeds
cumulativePrice = 0 + (1e18 * 31_536_000) = 3.1536e25
newAverage = (0 + 1e18 * 31_536_000) / (0 + 31_536_000) = 1e18
// Update SUCCESS ✓

// PHASE 2: 10 years pass, market moves 10% higher
elapsedTime = 315_360_000 seconds (10 years)
currentPrice = 1.1e18 (market price increased 10%)
cumulativePrice = 3.1536e25 (frozen from last update)
averagePrice = 1e18 (frozen from last update)
maxSlippage = 5%

// Slippage check happens FIRST (using OLD average)
upperBound = 1e18 * 1.05 = 1.05e18
lowerBound = 1e18 * 0.95 = 0.95e18
currentPrice = 1.1e18 > upperBound

// Update rejected! Function returns false
// State remains frozen - cumulativePrice and averagePrice NEVER update
```

The deadlock mechanism:

1. Oracle runs for extended period (weeks/months), accumulating large `cumulativePrice`
2. Market experiences legitimate volatility, price moves > `maxSlippage` from current average

3. Slippage check fails → function returns false
4. State is NOT updated: `cumulativePrice` and `averagePrice` stays frozen
5. Even after extended time passes, the frozen cumulative weight dominates any new time period
6. The calculated average cannot catch up to the market price
7. Slippage check in `validatePrice` is not accurate

Another way this can get frozen without having to wait for `cumulativePrice` to grow big

Long-term price moves can permanently halt protocol actions that rely on oracle validation. If market moves beyond `maxSlippage`, causing DoS for any consumer. `updatePrice()` simply returns false when the new price deviates beyond `maxSlippage`, leaving the snapshot unchanged. If the real price shifts permanently, the oracle never updates again and `validatePrice()` will keep failing, blocking buybacks or exits that depend on it.

Impact Explanation

`updatePrice()` returns false indefinitely, `cumulativePrice` and `averagePrice` stays frozen, `validatePrice` is going to work with an inaccurate slippage check, returning bad prices as valid and good prices as bad.

Likelihood Explanation

This is expected to occur naturally

Recommendation

The root cause is that slippage validation occurs against the **stale** average price before state updates, creating a deadlock when the oracle falls behind market movements.

Limit the TWAP window to something that you are ok with (30 minutes, 1 day, 1 month...). Add an emergency mechanism to reset the oracle when deadlocked.

Proof of Concept

[View detailed Proof of Concept](#)

Low Risk and Informational Issues

For this audit, 28 QA reports were submitted by wardens compiling low risk and informational issues. The [QA report highlighted below](#) by `rzizah` received the top score from the judge. 29 Low-severity findings were also submitted individually, and can be viewed [here](#).

The following wardens also submitted QA reports: [Oxbrett8571](#), [Oxcode](#), [Oxki](#), [Oxnija](#), [Oxterrah](#), [bamOx7](#), [freescor](#), [gaving101](#), [Ghamuop](#), [happykilling](#), [I1iveF0rTh1Sh1t](#).

[init_state](#), [jerry0422](#), [johnyfwesh](#), [K42](#), [Kazopl](#), [legat](#), [lioblaze](#), [lionking777](#), [mibunna](#), [mrMorningstar](#), [MRSHEEP](#), [piki](#), [PillarsOfLight](#), [recr4sh](#), [samueljbn](#), and [v12](#).

ID	TITLE	SEVERITY
L-01	Slippage protection in V3 swaps could be bypassed in some market conditions	LOW
L-02	Converting to V3 without a current V2 pool could be front-run leading to burning OLAS and loss of funds	LOW
L-03	Price could be manipulated in first 1800 seconds allowing attackers to influence liquidity operations for newly deployed pools	LOW
L-04	Ineffective slippage in LiquidityManagerCore due to reliance on spot prices instead of TWAP prices	LOW
L-05	TWAP is effectively a spot price in Uniswap V2 oracle	LOW
L-06	Registry address changes can lock user incentives	LOW
L-07	Incorrect slippage calculation doubles protection threshold	LOW
L-08	Precision loss in value comparison may lead to suboptimal liquidity optimization	LOW
L-09	Precision loss in donation distribution across service units	LOW
L-10	Slippage validation restricts minimum slippage to 1%, preventing sub-1% configurations	LOW
L-11	Invalid reward distribution configuration locks user funds and prevents unstaking	LOW
L-12	Precision loss in reward calculation causes wei discrepancies in proportional distribution	LOW
L-13	Checkpoint function becomes permanently unusable if not called within MAX_EPOCH_LENGTH	LOW

ID	TITLE	SEVERITY
[L-1] 4	Missing validation for maxSlippage parameter allows values exceeding maximum BPS limit	LOW
[L-1] 5	Missing slippage validation allows values exceeding 100% maximum limit	LOW

[L-01] slippage protection in V3 swaps could be bypassed in some market conditions

The `BuyBackBurner.sol` contract implements a buyback mechanism that swaps various tokens for OLAS tokens. For Uniswap V3 swaps, the contract performs price validation before executing the swap but lacks proper slippage protection during the actual swap execution.

The issue occurs in the `_buyOLAS()` function for V3 swaps where price validation is performed before the swap:

```
// Apply slippage protection
ILiquidityManager(liquidityManager).checkPoolAndGetCenterPrice(pool);

// Perform swap to OLAS
olasAmount = _performSwap(secondToken, secondTokenAmount,
    feeTierOrTickSpacing);
```

The root cause is that the pre-swap price validation only checks if the current price deviates from TWAP within acceptable bounds, but doesn't account for several critical scenarios:

1. **Low liquidity at active tick:** When there's insufficient liquidity at the current price tick, large swap amounts can cause significant price movement, resulting in the contract receiving far fewer OLAS tokens than expected not respecting maxslippage.
2. **Just-In-Time (JIT) liquidity manipulation:** Liquidity providers can move their liquidity to desirable price ranges just before the swap, creating artificial price levels that appear correct but result in poor execution.
3. **Lack of post-swap validation:** The V3 implementation has no mechanism to verify that the actual received amount matches expectations.

Impact: The contract can suffer significant financial losses when executing large swaps in low liquidity pools.

Scenarios: While there could be enough liquidity in the ranges that respect the maxslippage there is a case that

1. Attacker identifies a buyback call where
 1. low liquidity in the manipulated tick without surpassing the maxslippage so he manipulate the price near to maxslippage.
 2. Make sure that the going forward ranges have no liquidity (liquidity is present only the opposite direction tick ranges).
 3. leaving like 1 wei in the ticks
 4. add liquidity in very far a way price.
 5. ticks will keep shifting with out reverting due to lack of liquidity for the ticks.
2. when the buyback is called it will keep shifting ticks as there is no liquidity in the current tick causing severe price impact without reverting as the amountOutMinimum = 1

Numerical example

- Current price at active tick is 3000
- Twap price is 2990

With the overall market direction for the price to go up (most of the liquidity are still in the lower price ranges ex: <3010), And liquidity at prices above 3010 is few

- Attacker will manipulate the price pushing it to get near 3019 where the twap price still passes.
- Attacker will then leave dust liquidity in the tick and add JIT liquidity in a very high price tick.
- The ticks will keep shifting until either filling the swap or reach max-tick or reach the slippage and sense the slippage is 1wei the swap will execute with a very high price.
 - more on how ticks shifting works on uniV3 [here](#)
- Attacker will then swap back the funds stolen at the reasonable price after the buyback call completing the sandwich attack.

Recommended mitigation

Implement proper slippage protection for V3 swaps by:

1. Calculate expected output from quoter contract and apply slippage according to the amounts out: More on how to properly use quoter [here](#)
2. Add post-swap price validation:

```

// Perform swap to OLAS
olasAmount = _performSwap(secondToken, secondTokenAmount,
feeTierOrTickSpacing);

++                // Apply slippage protection

```

```
++
ILiquidityManager(liquidityManager).checkPoolAndGetCenterPrice(pool);
```

This approach ensures both price manipulation protection and protection against poor execution due to liquidity impacts, addressing both the pre-swap and post-swap validation requirements.

[L-02] Converting to V3 without a current V2 pool could be front-run leading to burning OLAS and causes a loss of funds

The `convertToV3()` function in `LiquidityManagerCore.sol` allow direct convert to v3 with no prior v2 pool that could be frontrun leading to loss of funds.

Looking at line 307 in the `convertToV3()` function:

```
if (v2Pool != 0) {
    // Remove liquidity from V2 pool
    _checkTokensAndRemoveLiquidityV2(tokens, v2Pool);
}
```

This check if there is a v2 pool to convert from it however in a case where a direct transfer were made (creating a v3 without v2 exist) the call of transferring the funds could be followed with a call to `collectFees()` frontrunning `convertToV3` call, as `collectFees()` is a permission-less function collecting fees and burn OLAS tokens so the transferred OLAS tokens will be burned before being provided as a liquidity in a v3 pool

1. **Monitoring for token transfers:** Attackers can monitor the blockchain for direct token transfers to the LiquidityManager contract address that where made to latter call `convertToV3` to create v3 pool through `liquiditymanagercore`
2. **Back-running the conversion:** When they detect a legitimate user sending tokens to the contract he back run the transfer and front-run the `convertToV3()` call
3. The attacker calls `collectFees()` before the legitimate user can call `convertToV3()`.
4. **Burning OLAS tokens:** the OLAS balance in the contract due to direct transfer, will be burn from `collectFees` call

```
File: LiquidityManagerCore.sol
835:
836:         // Manage collected fees: burn OLAS, transfer another token
837:@>         _manageUtilityAmounts(tokens, MAX_BPS, true);
838:
839:         emit PositionFeesCollected(pool, positionId, tokens,
amounts);
```

Impact

Greiving attack burning the OLAS tokens that is to be added to a v3 pool

1. **Token burning attacks:** Attackers can force the burning of OLAS tokens that the owner intended to use for liquidity provision

Recommended mitigation steps

- When handling direct transfer to provide liquidity with out previous v2 use EIP-7702 making an atomic call direct tokens transfer followed by calling `convertToV3` in the EIP-7702 smartcontract.
- **Require function calls for token provision:** Modify the contract to only accept tokens through explicit function calls rather than direct transfers (could be in the same `convertV3` function or a different function especially handling creating V3 pools with being in a v2 pool).

[L-03] Price could be manipulated in 1st 1800 seconds that attackers can influence liquidity operations for newly deployed pools that has few observations

The `LiquidityManagerCore.sol` contract implements price validation logic in the `checkPoolAndGetCenterPrice()` function. When the TWAP (Time-Weighted Average Price) oracle fails to provide valid data, the contract falls back to using the current `slot0` price from the Uniswap V3 pool, which can be easily manipulated by attackers.

The vulnerability occurs in two key locations where `_getPriceAndObservationIndexFromSlot0()` is called directly:

1. **Line 342:** In `_increaseLiquidity()` function
2. **Line 389:** In `_decreaseLiquidity()` function
3. **Line 1123:** In `checkPoolAndGetCenterPrice()` as a fallback when TWAP fails

The root cause is that the contract relies on the `slot0` price as a fallback mechanism when TWAP validation fails. However, the `slot0` price can be manipulated through large trades that temporarily move the pool price away from its true market value.

The `checkPoolAndGetCenterPrice()` function attempts to validate prices by comparing TWAP against spot prices, but has a case:

```
// If the call has failed - observe was not successful, meaning the pool
has not have enough activity yet
if (!success) {
    return centerSqrtPriceX96; // Returns manipulable slot0 price
}
```

More on oracle and when observe reverts found [here](#)

Impact

1. **Liquidity manipulation attacks:** An attacker can execute large trades to manipulate the slot0 price, frontrunning the trigger of liquidity operations (increase/decrease) at the manipulated price, causing the contract to provide liquidity at unfavorable rates or withdraw liquidity at sub-optimal prices.
2. **TWAP oracle failure exploitation:** When the TWAP oracle fails (due to insufficient observations), the contract automatically falls back to the manipulable slot0 price, creating a predictable attack vector (**all newly deployed pools that to be used in the contract will be vulnerable to oracle price manipulations leading to wrong liquidity provisioning and a loss of funds**).

Attack sequence:

1. Attacker identifies a pool where its newly deployed with low observation than so he easily manipulate the spot price with twap securing or any slippage protection for the protocol.
2. Attacker frontrun liquidity operations executes a large trade to move the pool price significantly away from fair market price
3. Liquidity operations (increase/decrease) on the LiquidityManager is called
4. Contract uses the manipulated slot0 price instead of fair market price
5. Attacker profits from the price discrepancy while the contract suffers huge losses as price has been manipulated.

Recommended mitigation steps

1. **Remove slot0 fallback:** Eliminate the fallback to slot0 price when TWAP fails. Instead, revert the transaction if TWAP validation cannot be completed.
2. **Or Implement external oracle for price validation:** Add additional price validation checks that compare against external price feeds.

[L-04] Ineffective slippage in liquiditymanagercore due to reliance on spot prices instead of TWAP prices

The `LiquidityManagerCore.sol` contract implements slippage protection mechanisms in the `_decreaseLiquidity()` and `_increaseLiquidity()` functions, but these protections are ineffective because they rely on spot prices from slot0 rather than the validated TWAP prices that the contract is designed to use for price validation.

The issue occurs in two critical locations:

1. **Line 359-360:** In `_decreaseLiquidity()` function:

```
amountsMin[0] = amountsMin[0] * (MAX_BPS - maxSlippage) / MAX_BPS;  
amountsMin[1] = amountsMin[1] * (MAX_BPS - maxSlippage) / MAX_BPS;
```

2. Line 417-418: In `_increaseLiquidity()` function:

```
aMin[0] = inputAmounts[0] * (MAX_BPS - maxSlippage) / MAX_BPS;  
aMin[1] = inputAmounts[1] * (MAX_BPS - maxSlippage) / MAX_BPS;
```

The root cause is that these slippage calculations use amounts derived from spot prices obtained via `_getPriceAndObservationIndexFromSlot0()`, which can be at the near maxslippage. While the contract has a sophisticated TWAP validation system in `checkPoolAndGetCenterPrice()`, this validation is performed but the results are not used for the actual slippage calculations after swap.

The contract flow demonstrates the inconsistency:

1. `checkPoolAndGetCenterPrice()` validates TWAP vs spot price deviation
2. The validation passes and returns a validated center price
3. However, `_decreaseLiquidity()` and `_increaseLiquidity()` call `_getPriceAndObservationIndexFromSlot0()` directly, bypassing the TWAP validation
4. Slippage calculations are performed using these potentially manipulable spot prices

Impact scenarios:

1. **maxslippage isn't applied correctly:** in some cases where there is a few liquidity in the traded tick price could move so the real maxslippage applied is `maxslippage + 10% (twapvalidation)` and not `maxslippage.ofit`

Recommended mitigation steps

1. **Use validated TWAP price for slippage calculations:** Modify `_decreaseLiquidity()` and `_increaseLiquidity()` to use the validated center price from `checkPoolAndGetCenterPrice()` instead of calling `_getPriceAndObservationIndexFromSlot0()` directly.

[L-05] TWAP is a spot price for uniV2 oracle

Uniswap V2 twap price is the tradeprice

```
File: UniswapPriceOracle.sol  
78:         // Calculate cumulative prices  
79:         uint256 cumulativePrice = cumulativePriceLast + (tradePrice *  
elapsedTime);
```

```

80:
81:         // Calculate the TWAP for OLAS in terms of native token
82:         uint256 timeWeightedAverage = (cumulativePrice -
cumulativePriceLast) / elapsedTime;
83:

```

Looking at the function

Applying the value of cumulativeprice in timeweightdaverage :

```

timeWeightedAverage = ((cumulativePriceLast + (tradePrice * elapsedTime))
- cumulativePriceLast) / elapsedTime
timeWeightedAverage = (tradePrice * elapsedTime) / elapsedTime
timeWeightedAverage = tradePrice

```

Impact

TWAP Price is the spot price.

Recommendation

Re-implement the TWAP calculations for uniV2 oracle by getting a previous cumulative and calculate the price by taking the average between them interval could be 1800

Could be something like this

```

price = uint224((recentPriceCumulative - snapshotPrice0Cumulative) /
timeElapsed);

```

[L-06] Registry address changes can lock user incentives

The changeRegistries() function in Tokenomics.sol allows updating registry addresses without ensuring that pending incentives are claimed first:

```

if (_agentRegistry != address(0)) {
    agentRegistry = _agentRegistry;
    emit AgentRegistryUpdated(_agentRegistry);
}

```

This creates an issue where if registry addresses are changed while owners have unclaimed incentives, those incentives become permanently inaccessible. The

`accountOwnerIncentives()` function uses the current registry addresses to verify ownership, so changing registries after incentives have been accumulated but before claiming will cause ownership verification to fail.

Impact: Users with pending incentives lose access to their rewards if registry addresses are changed before they claim their incentives.

Recommendation: Implement a mechanism to require all pending incentives to be claimed before registry addresses can be updated, or provide a migration mechanism to preserve user incentives across registry changes.

[L-07] Incorrect slippage calculation doubles protection threshold

The `_buyOLAS()` function in `BuyBackBurner.sol` applies slippage protection incorrectly by using the trade price instead of the TWAP price for validation:

```
uint256 lowerBound = (previousPrice * (100 - maxSlippage)) / 100;
uint256 upperBound = (previousPrice * (100 + maxSlippage)) / 100;
require(tradePrice >= lowerBound && tradePrice <= upperBound, "After swap
slippage limit is breached");
```

This creates a double slippage protection issue where:

1. The initial `validatePrice(maxSlippage)` check already validates against TWAP price with slippage
2. The post-swap validation uses the trade price (which can be significantly different from TWAP) against the same slippage bounds

Impact: During high-volume swaps or in low liquidity pools, the trade price can deviate significantly from the TWAP price, causing illegitimate swaps to succeed though they surpassed the max slippage.

Recommendation: Use the TWAP price for both pre and post-swap validation, or implement a separate, more lenient slippage threshold for trade price validation to account for temporary price deviations during large swaps.

This could be achieved by changing the order of the `require` making it to execute after the swap. This will ensure correct slippage.

[L-08] Precision loss in value comparison may lead to suboptimal liquidity optimization

The `value0InToken1()` function in `NeighborhoodScanner.sol` calculates the value of `token0` in terms of `token1` using an intermediate calculation that introduces unnecessary precision loss. The function performs the calculation in two steps:

```
uint256 intermediate = FullMath.mulDiv(amount, sqrtP, FixedPoint96.Q96);
return FullMath.mulDiv(intermediate, sqrtP, FixedPoint96.Q96);
```

This approach first calculates $\text{amount} * \text{sqrtP} / \text{Q96}$, then multiplies by sqrtP again and divides by Q96 . However, this intermediate step introduces rounding errors that could affect the accuracy of the value comparison in `_chooseMode()`.

The `_chooseMode()` function uses this value to determine whether to optimize for high or low ticks by comparing `value0InToken1(amounts[0], sqrtP)` with `amounts[1]`. Due to the precision loss in the intermediate calculation, the comparison may not accurately reflect the true relative values of the token amounts, potentially leading to suboptimal liquidity range selection.

For example, if the true value ratio is very close to 1:1, the precision loss could cause the function to choose the wrong optimization direction, resulting in less efficient use of available liquidity.

Recommendation: Calculate the value directly using the more precise formula:

```
return FullMath.mulDiv(amount, uint256(sqrtP) * sqrtP,
uint256(FixedPoint96.Q96) * FixedPoint96.Q96);
```

This eliminates the intermediate step and reduces cumulative rounding errors, providing more accurate value comparisons for liquidity optimization decisions.

- Check whether the price is larger or the amount and use the largest for `intermediate`.

[L-09] Precision loss in donation distribution across service units

The `_trackServiceDonations()` function in `Tokenomics.sol` uses integer division to distribute ETH donations equally across service units:

```
uint96 amount = uint96(amounts[i] / numServiceUnits);
```

When `amounts[i]` is not perfectly divisible by `numServiceUnits`, the remainder is truncated due to Solidity's integer division behavior. This results in precision loss where the total distributed amounts across all units in a service will be less than the original donation amount.

For example, if a service receives $100\text{e}18$ wei in donations and has 3 units, each unit receives $33\text{e}17$ wei ($33 * 3 = 999999999999999999$), with 1 wei lost to truncation. Over time and

multiple donation events, this precision loss accumulates and results in component/agent owners receiving slightly less rewards than they should.

Impact: Component and agent owners receive marginally reduced incentives due to systematic precision loss in donation distribution calculations.

Recommendation: Implement a distribution mechanism that accounts for remainders, such as distributing the remainder to a specific unit or using a more sophisticated allocation algorithm that preserves the total donation amount.

[L-10] Slippage validation restricts minimum slippage to 1%, preventing sub-1% slippage configurations

The `LiquidityManagerETH.sol`, `LiquidityManagerOptimism.sol` contracts includes a slippage validation mechanism in the `_checkTokensAndRemoveLiquidityV2()` function that restricts the minimum slippage to 1%.

```
if (!IOracle(oracleV2).validatePrice(maxSlippage / 100)) {  
    revert SlippageLimitBreached();  
}
```

The issue is that the `maxSlippage` parameter is divided by 100 before being passed to the oracle's `validatePrice()` function. This means that any `maxSlippage` value less than 100 (representing less than 1% slippage) will be converted to 0 when passed to the oracle, effectively preventing the use of sub-1% slippage tolerances.

This restriction could be problematic in case the `maxslippage` is set to a value $< 1\%$.

[L-11] Invalid reward distribution configuration locks user funds and prevents unstaking

The `StakingBase.sol` contract validate rewardinfo in case custom in `_getRewardReceiversAndAmounts()` function that allows users to set reward distribution configurations. However if something happen to the address with custom configuration there is no way for the user to neither claim nor unstake leading to a DOS freeze of funds as there is no function to change the address.

The issue occurs in the reward distribution validation logic:

```
} else if (uint160(rewardDistributionInfo >> 8) != 0) {  
    // Make sure upper bits do not have any value if reward distribution  
    type is not Custom  
    revert NonZeroValue();  
}
```

```
}
```

The problem is that this validation checks for non-zero values when the reward distribution type is Custom. A user can set a Custom reward distribution type (value 3) but after some time the distribute could have some errors or the user mistakenly set the distributor address, which creates an invalid state.

Impact

1. **Fund lockup:** Users who accidentally set an invalid Custom reward distribution configuration cannot unstake their service or claim rewards, effectively locking their funds permanently.
2. No function to correct the address of the distributor

Impact: While this represents a user experience and fund accessibility issue rather than a direct security vulnerability, it can result in permanent loss of user funds due to configuration errors.

Recommendation: However this is a user mistake, implement a function for a user to change the distributor address.

[L-12] Precision loss in reward calculation causes wei discrepancies in proportional distribution

The `calculateStakingReward()` function in `StakingBase.sol` suffers from precision loss during reward calculation when total allocated rewards exceed available rewards, leading to small wei discrepancies in the final reward distribution.

The issue occurs in the `_calculateStakingRewards()` function where proportional reward adjustments are made:

```
// Calculate the updated reward
updatedReward = (eligibleServiceRewards[i] * lastAvailableRewards) /
totalRewards;
```

When `totalRewards > lastAvailableRewards`, the contract performs proportional adjustments to fit rewards within the available budget. However, this division operation causes precision loss that accumulates across multiple services, resulting in:

1. **Wei discrepancies:** Small amounts of rewards (typically 1-2 wei) are lost due to integer division truncation
2. **Inaccurate reward reporting:** The `calculateStakingReward()` function reports slightly different values than what would actually be received

Impact: While the precision loss is minimal (typically 1-2 wei per calculation), it should be taken into consideration if to be implemented in frontend or other calculations

Recommendation: add the leftover to the owner just as with calculations in checkpoint

```
        // If the reward adjustment happened to have small
leftovers, add it to the first service
        if (lastAvailableRewards > updatedTotalRewards) {
            updatedReward += lastAvailableRewards -
updatedTotalRewards;
        }
```

[L-13] Checkpoint function becomes permanently unusable if not called within MAX_EPOCH_LENGTH

The `checkpoint()` function in `Tokenomics.sol` includes a validation that prevents execution if too much time has passed since the last epoch end:

```
if (diffNumSeconds < curEpochLen || diffNumSeconds > MAX_EPOCH_LENGTH) {
    return false;
}
```

This creates a usability issue where if the checkpoint function is not called within `MAX_EPOCH_LENGTH` (approximately one year), it becomes permanently unusable. The function will always return `false` and never advance the epoch counter, effectively freezing the tokenomics system with any leftover incentives.

Impact: If the checkpoint function is not called within the maximum epoch length timeframe, the entire tokenomics system becomes permanently stuck, preventing epoch progression and incentive distribution.

Recommendation: Implement a mechanism to handle extended periods without checkpoints, such as allowing the function to proceed with the actual elapsed time or providing an emergency recovery mechanism for the DAO to reset the system state.

[L-14] Missing validation for `maxSlippage` parameter allows values exceeding maximum BPS limit

The `changeMaxSlippage()` function in `LiquidityManagerCore.sol` at line 255 lacks proper validation for the `newMaxSlippage` parameter. While the function checks for zero values, it fails to validate that the provided slippage value does not exceed the maximum allowed BPS (Basis Points) limit of 10,000.

The current validation only includes:

```
// Check for zero value
if (newMaxSlippage == 0) {
    revert ZeroValue();
}
```

However, it's missing the upper bound check that exists in other functions like `initialize()`:

```
// Check for max value
if (_maxSlippage > MAX_BPS) {
    revert Overflow(_maxSlippage, MAX_BPS);
}
```

This oversight allows `maxSlippage` to values greater than 10,000, which could lead to unexpected behavior in slippage calculations throughout the contract. Since slippage values are used in percentage calculations with the `MAX_BPS` constant as the denominator, values exceeding this limit could result in:

Recommendation: Add the missing upper bound validation to the `changeMaxSlippage()` function:

```
// Check for max value
if (newMaxSlippage > MAX_BPS) {
    revert Overflow(newMaxSlippage, MAX_BPS);
}
```

[L-15] Missing slippage validation allows values exceeding 100% maximum limit

The `UniswapPriceOracle.sol` contract

The `maxSlippage` isn't enforced to be less than 100 in `uniswappriceoracle.sol` as with `balancer`

Recommendation: implement the missing validation in the constructor:

```
require(_maxSlippage < 100, "Slippage must be less than 100%");
```

Disclosures

C4 audits incentivize the discovery of exploits, vulnerabilities, and bugs in smart contracts. Security researchers are rewarded at an increasing rate for finding higher-risk issues. Audit submissions are judged by a knowledgeable security researcher and disclosed to sponsoring developers. C4 does not conduct formal verification regarding the provided code but instead provides final verification.

C4 does not provide any guarantee or warranty regarding the security of this project. All smart contract software should be used at the sole risk and responsibility of users.